Modul Pelatihan Flutter 3 Hari

Nov 1, 2026 · @omayib

Pendahuluan

Setelah 3 hari (08.00–16.00 WIB), peserta mampu membangun aplikasi Flutter yang mengambil data dari API, mengelola state, dan mem-build-nya menjadi file .apk.

Sasaran peserta: mahasiswa atau pengembang pemula yang sudah memahami logika pemrograman dasar (percabangan, perulangan) di bahasa apa pun.

Prasyarat perangkat: laptop RAM minimal 8 GB, ruang kosong 15 GB, koneksi internet stabil, dan HP Android dengan *USB debugging* (opsional, emulator bisa dipakai).

Capaian pembelajaran

1. Menulis kode Dart dengan variabel, fungsi, dan kelas.
2. Menyusun UI dengan widget Stateless dan Stateful.
3. Mengambil dan menampilkan data JSON dari API publik.
4. Membangun layout daftar, grid, dan adaptif, serta navigasi antarhalaman.
5. Mengelola state aplikasi dengan Provider.
6. Mem-build dan mempersiapkan aplikasi untuk dipublikasikan.

Jadwal harian

Waktu (WIB)	Kegiatan
08.00–09.45	Sesi 1
09.45–10.00	Rehat
10.00–12.00	Sesi 2
12.00–13.00	Ishoma
13.00–14.30	Sesi 3
14.30–16.00	Praktik / proyek

Cara memakai checkpoint HOTS

Setiap sesi diakhiri 3–5 pertanyaan *Higher Order Thinking Skills* berlevel Bloom C4 (menganalisis), C5 (mengevaluasi), atau C6 (mencipta). Gunakan 10–15 menit di akhir sesi: peserta menjawab, lalu memaparkan jawabannya. Jawaban tidak dinilai benar-salah semata, tetapi dari ketepatan alasan.

Bagi fasilitator, apabila sebagian besar peserta kesulitan menjawab pertanyaan C4, ulas kembali konsep inti sebelum lanjut ke sesi berikutnya.

Hari 1: Fondasi Dart & Pengenalan Flutter

Hari pertama membekali peserta dengan bahasa Dart dan cara berpikir "semuanya adalah widget", ditutup dengan membuat halaman login atau profil.

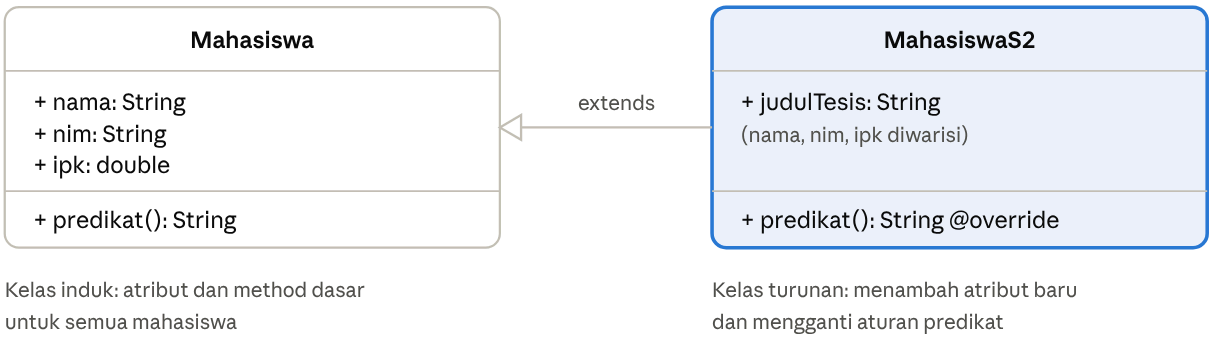
Sesi 1: Pengenalan Bahasa Dart

Tujuan: peserta mampu menulis program Dart yang memakai variabel, fungsi, dan kelas sederhana.

Materi pokok

- Tipe data dan deklarasi: `var`, `final`, `const`, `int`, `double`, `String`, `bool`, `List`, `Map`.
- *Null safety*: tipe nullable (`String?`), operator `?.`, `??`, dan `!`.
- Fungsi: parameter posisional, parameter bernama (`{required}`), *arrow function*, fungsi sebagai nilai.
- OOP dasar: kelas, konstruktor, *named constructor*, pewarisan (`extends`), dan *override*.

MahasiswaS2 mewarisi Mahasiswa dan mengganti predikat()

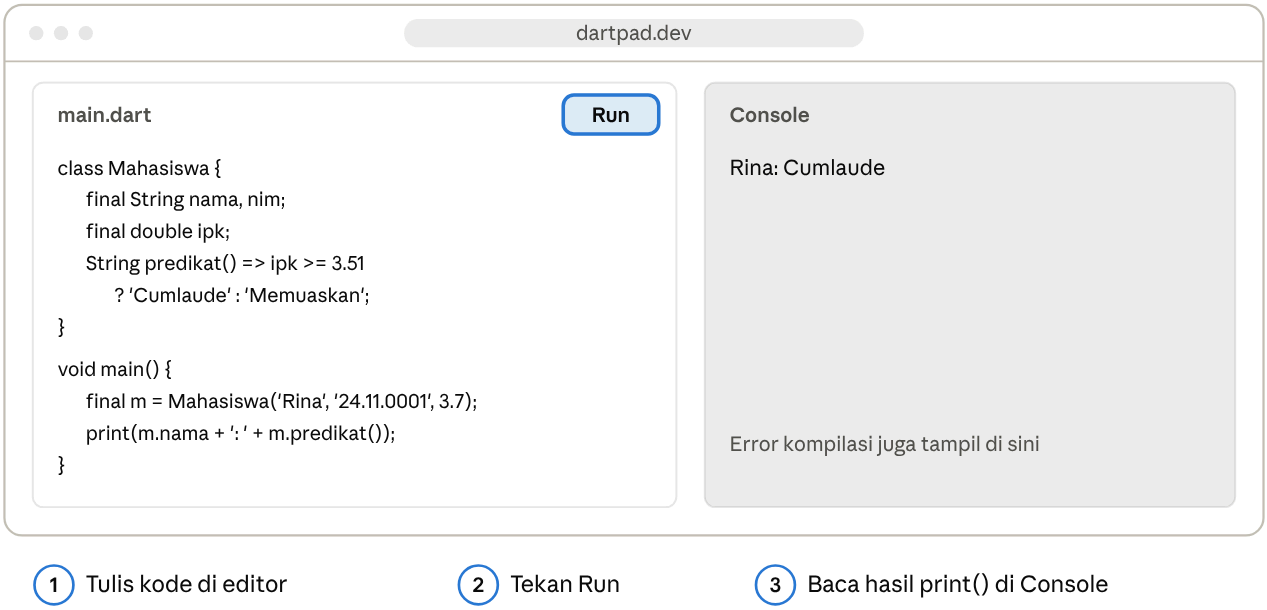


Konsep OOP · diagram kelas pewarisan

Panah bersegitiga kosong berarti "mewarisi": `MahasiswaS2` otomatis punya `nama`, `nim`, dan `ipk`, lalu menimpa `predikat()` dengan aturannya sendiri.

Aktivitas: di DartPad, peserta membuat kelas `Mahasiswa` (nama, NIM, IPK) dengan method `predikat()` yang mengembalikan "Cumlaude", "Sangat Memuaskan", atau "Memuaskan".

DartPad: tulis kode, tekan Run, baca hasil di Console



Wireframe sesi · DartPad

Sepanjang sesi ini peserta cukup bekerja di satu layar browser: kode di kiri, hasil di kanan, tanpa instalasi apa pun.

Tutorial langkah demi langkah

Langkah 1 — Buka DartPad. Buka dartpad.dev di browser. Hapus kode contoh sampai tersisa fungsi `main()` kosong. Semua latihan sesi ini dijalankan dengan tombol **Run**.

Langkah 2 — Variabel dan tipe data. Ketik lalu jalankan:

```
void main() {
  var kota = 'Yogyakarta';      // tipe ditebak: String
  int umur = 21;
  double ipk = 3.65;
  bool aktif = true;
  final angkatan = 2024;        // nilai ditetapkan sekali, saat runtime
  const pi = 3.14;              // konstanta saat kompilasi
  List<String> hobi = ['coding', 'musik'];
  Map<String, int> nilai = {'Dart': 90, 'Flutter': 85};

  print('$kota, umur $umur, IPK $ipk');
  print('Hobi pertama: ${hobi[0]}, nilai Dart: ${nilai['Dart']}');
}
```

Coba ubah `angkatan = 2025;` di baris bawahnya dan perhatikan pesan error-nya.

Langkah 3 — Null safety. Tambahkan di dalam `main()`:

```
String? namaPanggilan;                    // boleh null
print(namaPanggilan?.length);             // null, tidak crash
print(namaPanggilan ?? 'Belum diisi');    // nilai cadangan
```

Lalu coba `print(namaPanggilan!.length);` dan amati error saat dijalankan. Diskusikan mengapa `!` berbahaya.

Langkah 4 — Fungsi. Tulis di luar `main()`, lalu panggil dari `main()`:

```
int tambah(int a, int b) => a + b;

String sapa({required String nama, String salam = 'Halo'}) {
  return '$salam, $nama!';
}

// di main():
print(tambah(3, 4));
print(sapa(nama: 'Budi'));
print(sapa(nama: 'Sari', salam: 'Selamat pagi'));
```

Langkah 5 — Kelas dan konstruktor. Buat kelas `Mahasiswa` di luar `main()`:

```
class Mahasiswa {
  final String nama;
  final String nim;
  final double ipk;

  Mahasiswa(this.nama, this.nim, this.ipk);

  String predikat() {
    if (ipk >= 3.51) return 'Cumlaude';
    if (ipk >= 3.01) return 'Sangat Memuaskan';
    return 'Memuaskan';
  }
}

// di main():
final m = Mahasiswa('Rina', '24.11.0001', 3.7);
print('${m.nama}: ${m.predikat()}');
```

Langkah 6 — Pewarisan dan override. Tambahkan kelas turunan:

```
class MahasiswaS2 extends Mahasiswa {
  final String judulTesis;
  MahasiswaS2(String nama, String nim, double ipk, this.judulTesis)
      : super(nama, nim, ipk);

  @override
  String predikat() => ipk >= 3.75 ? 'Cumlaude' : super.predikat();
}
```

Buat satu objek `MahasiswaS2` ber-IPK 3.6 dan bandingkan predikatnya dengan objek `Mahasiswa` ber-IPK sama.

Langkah 7 — Tantangan mandiri (10 menit). Simpan beberapa objek mahasiswa dalam `List<Mahasiswa>`, lalu cetak hanya yang berpredikat Cumlaude memakai `where()`.

Checkpoint HOTS

1. (C4) Bandingkan `final` dan `const`. Dalam situasi apa sebuah nilai boleh `final` tetapi tidak boleh `const`? Berikan contoh dari aplikasi nyata.
2. (C4) Dart menerapkan *null safety*. Analisis jenis bug apa yang dicegah fitur ini dibanding bahasa tanpa null safety.

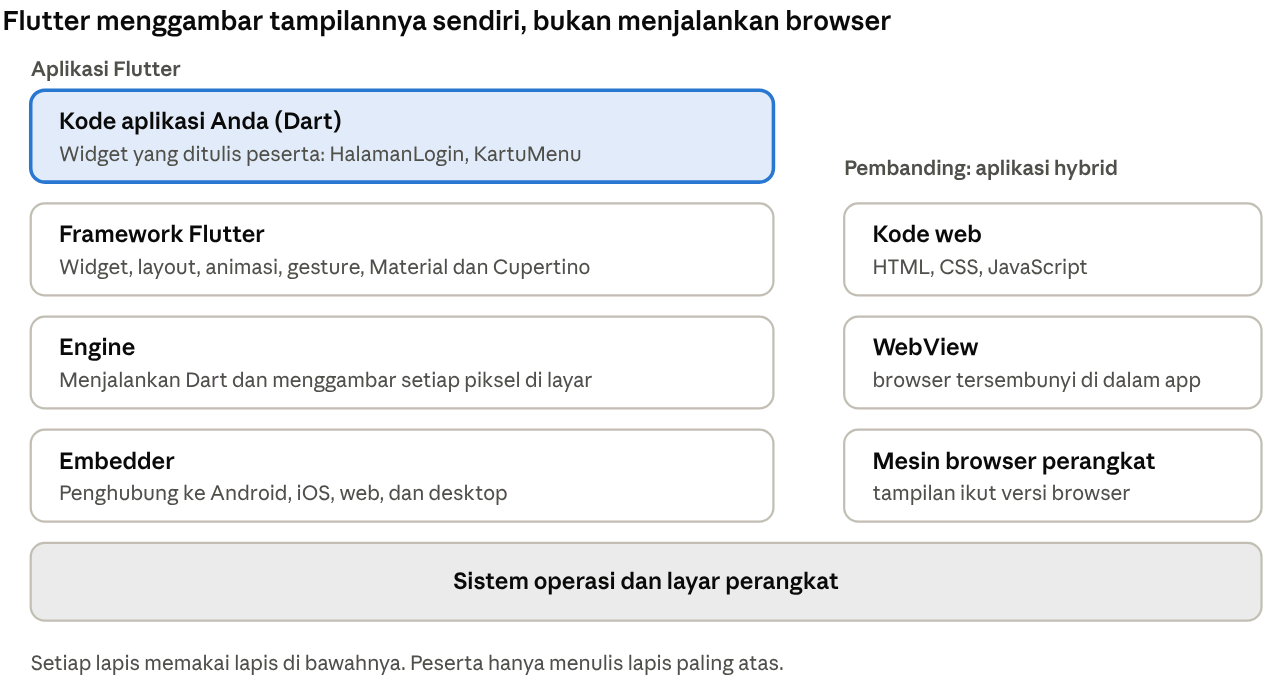
3. (C5) Seorang rekan memakai `!` di semua variabel nullable agar kodenya cepat jalan. Nilailah praktik ini: apa risikonya dan apa alternatif yang lebih aman?
4. (C6) Rancang hierarki kelas untuk sistem perpustakaan (Buku, Majalah, E-Book). Atribut dan method mana yang ditaruh di kelas induk, dan mana di kelas turunan? Jelaskan alasannya.

Sesi 2: Instalasi Flutter SDK & Konfigurasi IDE

Tujuan: setiap peserta berhasil menjalankan aplikasi contoh Flutter di emulator atau HP.

Materi pokok

- Arsitektur Flutter: Dart, *framework*, *engine*, dan *embedder*; perbedaannya dengan aplikasi native dan hybrid berbasis WebView.
- Instalasi Flutter SDK, pengaturan PATH, dan Android SDK.
- Membaca hasil `flutter doctor` dan memperbaiki masalah umumnya.
- Ekstensi Flutter/Dart di VS Code atau plugin di Android Studio; membuat emulator (AVD).
- `flutter create`, struktur folder proyek, `pubspec.yaml`, *hot reload* vs *hot restart*.

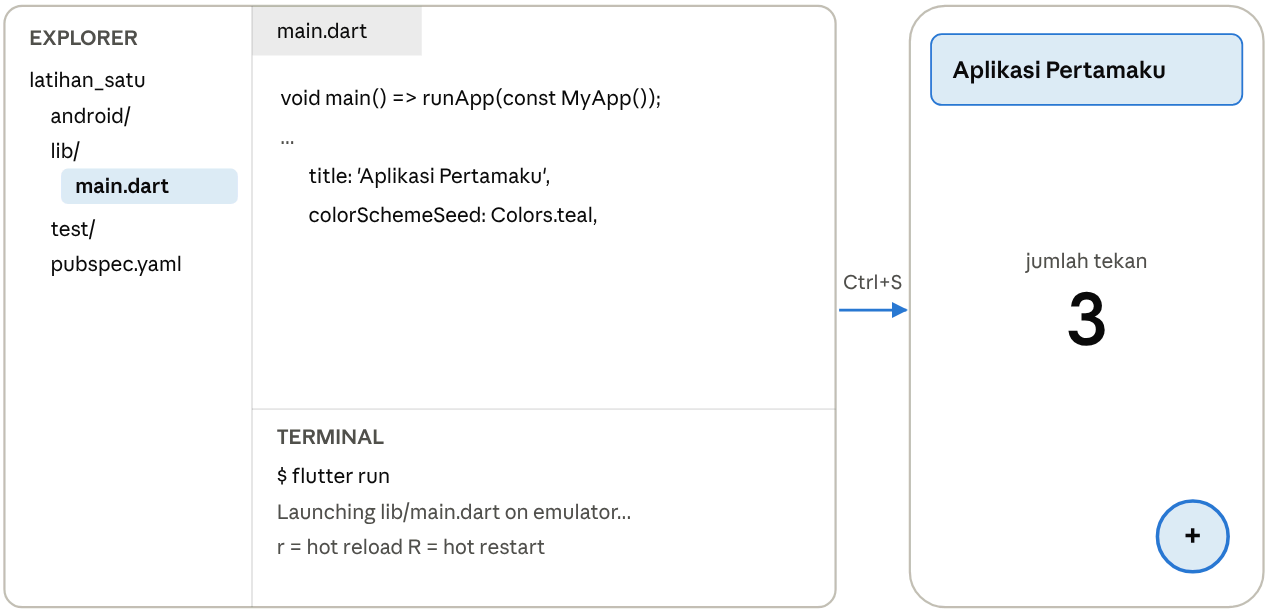


Konsep arsitektur · lapisan Flutter vs hybrid

Karena Engine menggambar sendiri setiap piksel, tampilan aplikasi Flutter sama di semua perangkat sedangkan aplikasi hybrid bergantung pada browser bawaan HP.

Aktivitas: membuat proyek `latihan_satu`, menjalankannya, lalu mengubah teks dan warna untuk merasakan hot reload.

Target sesi: latihan_satu berjalan di emulator



Ubah judul atau warna, simpan, dan layar emulator berubah tanpa angka penghitung kembali ke 0.

Wireframe sesi · VS Code dan emulator

Sesi dianggap selesai bila setiap peserta melihat layar seperti ini di laptopnya sendiri.

Tutorial langkah demi langkah

Catatan fasilitator: minta peserta mengunduh Flutter SDK, Android Studio, dan VS Code sehari sebelum pelatihan (total sekitar 3 GB) agar sesi ini tidak habis untuk menunggu unduhan. Ikuti petunjuk resmi di docs.flutter.dev/get-started/install bila ada perbedaan versi.

Langkah 1 — Unduh dan ekstrak Flutter SDK. Unduh file zip Flutter SDK versi *stable* untuk sistem operasi Anda. Ekstrak ke folder tanpa spasi dan tanpa hak admin, misalnya `C:\src\flutter` (Windows) atau `~/development/flutter` (macOS/Linux). Jangan taruh di `C:\Program Files`.

Langkah 2 — Tambahkan ke PATH.

- Windows: buka *Edit the system environment variables* → *Environment Variables* → pilih `Path` pada *User variables* → *New* → isi `C:\src\flutter\bin`.
- macOS/Linux: tambahkan baris berikut ke `~/.zshrc` atau `~/.bashrc`, lalu buka ulang terminal:

```
export PATH="$HOME/development/flutter/bin:$PATH"
```

Cek dengan membuka terminal baru dan menjalankan `flutter --version`.

Langkah 3 — Pasang Android Studio dan Android SDK. Instal Android Studio, jalankan *Setup Wizard* dengan pilihan *Standard*. Setelah selesai, buka *SDK Manager* → tab *SDK Tools* → centang **Android SDK Command-line Tools** → *Apply*.

Langkah 4 — Jalankan flutter doctor.

```
flutter doctor
flutter doctor --android-licenses   # ketik y untuk setiap lisensi
flutter doctor
```

Target: centang pada *Flutter*, *Android toolchain*, dan minimal satu IDE. Tanda silang pada *Visual Studio* atau *Xcode* boleh diabaikan bila tidak membuat aplikasi desktop atau iOS.

Langkah 5 — Siapkan IDE.

- VS Code: buka *Extensions* (Ctrl+Shift+X), instal ekstensi **Flutter** (ekstensi Dart ikut terpasang).
- Android Studio: *Settings* → *Plugins* → cari **Flutter** → *Install* → restart.

Langkah 6 — Siapkan perangkat uji. Pilih salah satu:

- Emulator: Android Studio → *Device Manager* → *Create Device* → pilih Pixel → pilih *system image* terbaru → *Finish* → tekan tombol play.
- HP fisik: *Pengaturan* → *Tentang ponsel* → ketuk *Nomor build* 7 kali → masuk *Opsi pengembang* → aktifkan *USB debugging* → sambungkan kabel dan izinkan di HP.

Cek dengan `flutter devices`; perangkat Anda harus muncul di daftar.

Langkah 7 — Buat dan jalankan proyek pertama.

```
flutter create latihan_satu
cd latihan_satu
flutter run
```

Di VS Code bisa juga: Ctrl+Shift+P → *Flutter: New Project* → *Application*, lalu tekan F5.

Langkah 8 — Kenali struktur proyek. Tunjukkan bersama peserta: `lib/main.dart` (kode utama), `pubspec.yaml` (nama aplikasi, dependensi, aset), `android/` (konfigurasi khusus Android), dan `test/`.

Langkah 9 — Rasakan hot reload. Saat aplikasi berjalan, buka `lib/main.dart`, ubah `title: 'Flutter Demo Home Page'` menjadi `'Aplikasi Pertamaku'` dan `Colors.deepPurple` menjadi `Colors.teal`, lalu simpan (Ctrl+S). Perhatikan angka penghitung tidak kembali ke 0. Sekarang tekan *hot restart* (Ctrl+Shift+F5 atau `R` di terminal) dan bandingkan.

Checkpoint HOTS

1. (C4) `flutter doctor` menampilkan tanda silang pada "Android toolchain" tetapi centang pada "Flutter". Uraikan kemungkinan penyebabnya dan urutan langkah diagnosisnya.
2. (C4) Mengapa perubahan di `initState()` tidak tampil dengan hot reload, tetapi tampil dengan hot restart? Kaitkan dengan cara Flutter menyimpan state.
3. (C5) Laptop peserta hanya ber-RAM 8 GB. Evaluasi pilihan berikut untuk menguji aplikasi: emulator Android, HP fisik via USB, atau Flutter Web di browser. Mana yang paling tepat dan apa kompromi masing-masing?
4. (C5) Bandingkan pendekatan Flutter (render sendiri lewat engine) dengan aplikasi hybrid berbasis WebView. Untuk aplikasi kasir UMKM, mana yang lebih Anda rekomendasikan dan mengapa?

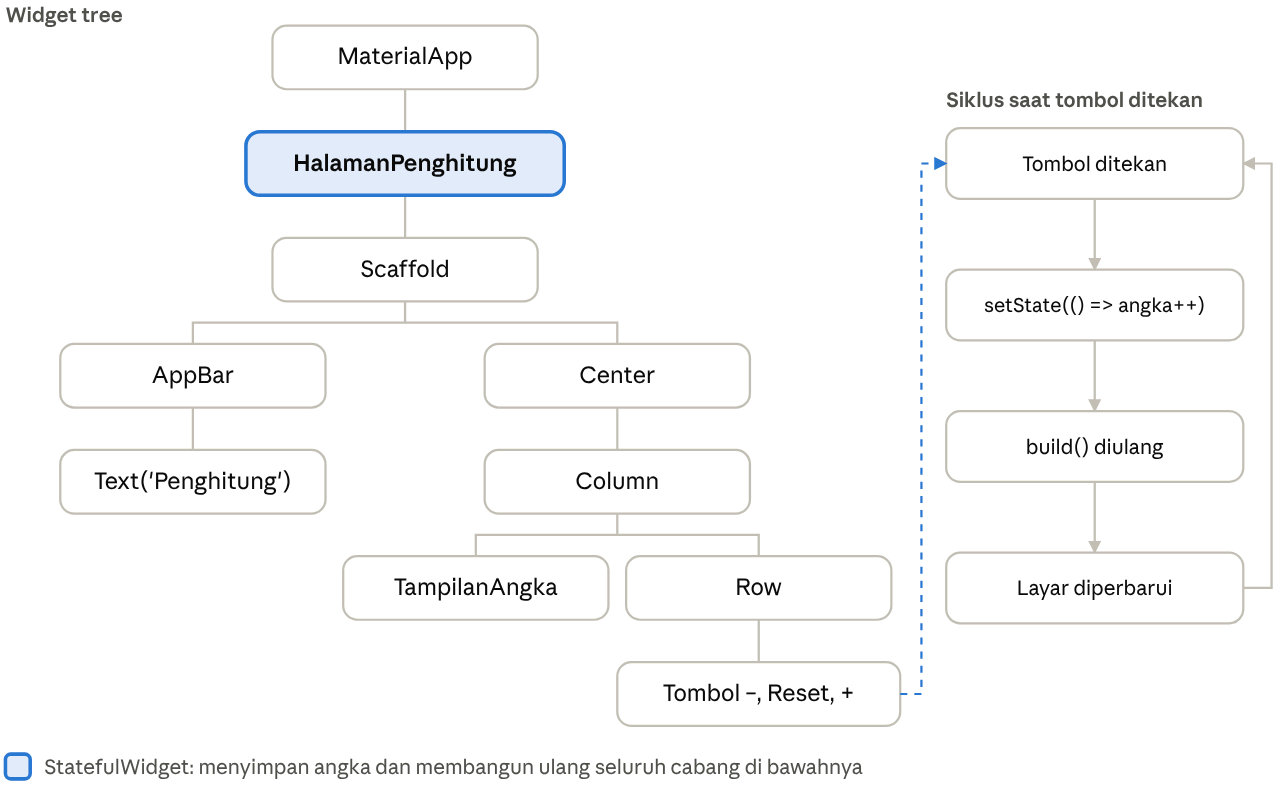
Sesi 3: Konsep Dasar Flutter — Everything is a Widget

Tujuan: peserta mampu membedakan dan memakai StatelessWidget dan StatefulWidget.

Materi pokok

- *Widget tree*: komposisi widget di dalam widget.
- Widget dasar: `MaterialApp`, `Scaffold`, `AppBar`, `Text`, `Container`, `Row`, `Column`, `Image`, `ElevatedButton`.
- StatelessWidget vs StatefulWidget; `setState()` dan siklus hidup (`initState`, `build`, `dispose`).
- Ekstraksi widget agar kode rapi dan dapat dipakai ulang, lalu memeriksa widget tree dengan Flutter Inspector di DevTools.

Semua adalah widget: penghitung tersusun dari pohon widget



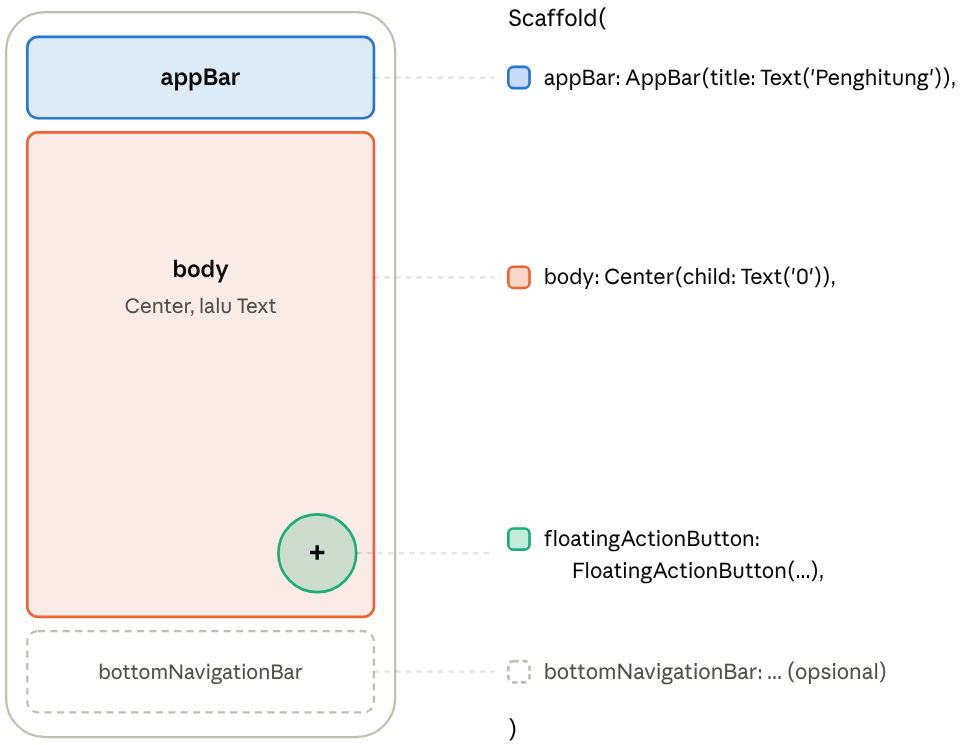
Konsep widget · widget tree dan siklus setState

Hanya `HalamanPenghitung` yang Stateful; widget lain cukup Stateless karena hanya menampilkan data yang diberikan dari atas.

Membaca Scaffold: mengisi slot, bukan mengakses komponen

Bagi pemula, menyusun UI lewat kode sering membingungkan karena seolah-olah kita "mengakses" appBar atau body yang sudah ada. Kenyataannya sebaliknya: `Scaffold` adalah denah halaman yang menyediakan slot-slot bernama, dan kita mengisi tiap slot dengan widget pilihan kita lewat parameter bernama di konstruktornya. Catatan: di tutorial, `AplikasiKu` hanya berisi `MaterialApp` (urusan tingkat aplikasi seperti tema dan navigasi), sedangkan `Scaffold` ada di `HalamanPenghitung` (urusan satu halaman).

Setiap parameter Scaffold mengisi satu area layar



Konsep widget · slot bernama pada Scaffold

Empat cara menjelaskannya ke peserta:

1. **Widget adalah kelas, slot adalah parameter bernama.** `Scaffold(appBar: ..., body: ...)` sama persis dengan memanggil `sapa(nama: 'Budi')` di Sesi 1, hanya saja nilai yang dikirim berupa widget. Jembatan ini penting karena peserta baru saja mempelajari parameter bernama di Dart.
2. **Analogi formulir.** `Scaffold` seperti formulir dengan kolom berlabel: "bagian atas", "isi", "tombol melayang". Kolom yang tidak diisi dibiarkan kosong dan tidak digambar. Hapus baris `floatingActionButton:` untuk menunjukkan tombolnya hilang.
3. **Setiap slot punya tipe yang diharapkan.** `body` menerima widget apa saja, sedangkan `appBar` hanya menerima `PreferredSizeWidget` (biasanya `AppBar`) karena tingginya harus diketahui. Coba isi `appBar: Text('Halo')` dan tunjukkan error merahnya: editor ikut menjaga peserta.
4. **Isi slot boleh bersarang.** `body: Center(child: Column(children: [...]))` berarti slot `body` diisi `Center`, lalu slot `child` milik `Center` diisi `Column`. Inilah yang membentuk widget tree pada diagram sebelumnya.

Ajarkan juga cara menemukan slot yang tersedia sebagai kebiasaan, bukan hafalan:

- **Ctrl+Space** di dalam kurung `Scaffold( )` menampilkan semua parameter yang bisa diisi.
- **Arahkan kursor** ke nama widget untuk melihat ringkasan dokumentasi dan tipe setiap parameter.
- **Ctrl+klik** `Scaffold` untuk membuka kode sumbernya, termasuk daftar lengkap parameter konstruktor.
- [api.flutter.dev](): cari "Scaffold class", lalu baca bagian *Constructors* dan *Properties*.
- **Flutter Inspector** (Langkah 7 tutorial): klik area di layar untuk melihat widget yang menggambarnya.

Slot Scaffold lain yang sering dipakai pemula:

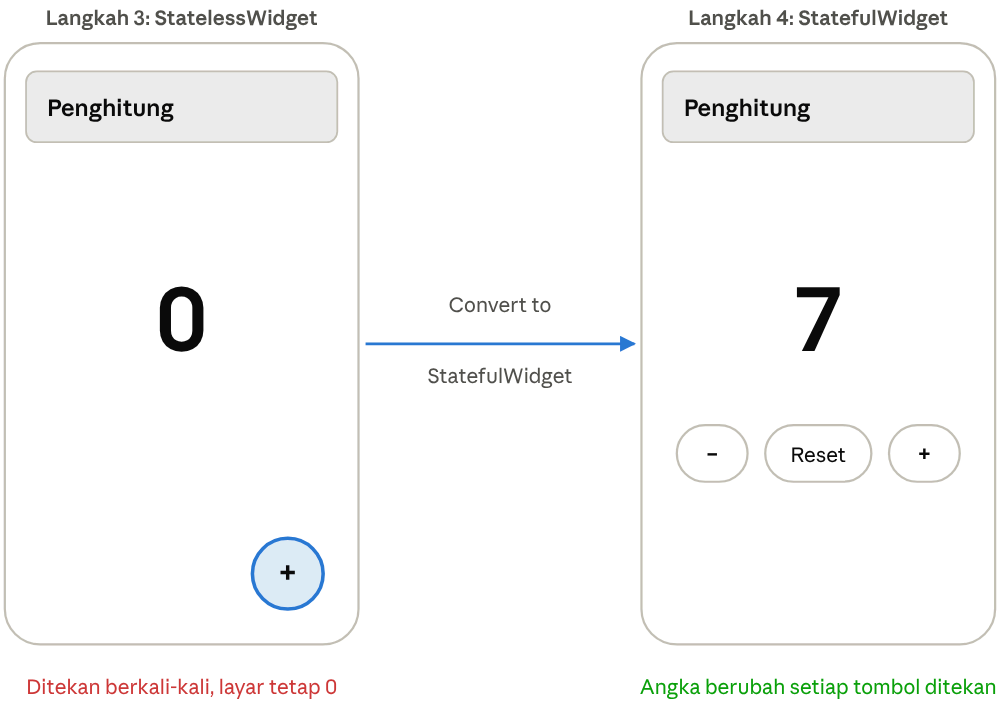
Slot	Fungsi	Muncul lagi di
`drawer` / `endDrawer`	Menu geser dari kiri/kanan; ikon hamburger muncul otomatis di AppBar	Tantangan mandiri
`bottomNavigationBar`	Navigasi di bagian bawah	Hari 2, Sesi 3
`bottomSheet`, `persistentFooterButtons`	Panel atau tombol yang menempel di bawah	Tantangan mandiri
`backgroundColor`	Warna latar halaman	Praktik Hari 1
`floatingActionButtonLocation`	Memindah FAB ke tengah bawah, kiri, dan seterusnya	Tantangan mandiri
`resizeToAvoidBottomInset`	Mengatur apakah halaman menyusut saat keyboard muncul	Praktik login (overflow)

Slot-slot itu juga punya slot sendiri. `AppBar`, misalnya, punya `title`, `leading` (ikon kiri), `actions` (ikon kanan, seperti badge keranjang di Hari 3), `centerTitle`, `backgroundColor`, dan `bottom` (untuk `TabBar`).

Latihan "isi slot" (10 menit). Beri peserta `Scaffold` kosong dengan tantangan: isi sebanyak mungkin slot tanpa melihat tutorial, hanya dengan Ctrl+Space dan hover. Pemenangnya peserta dengan slot terisi terbanyak tanpa error. Latihan ini melatih cara mencari tahu, bukan menghafal.

Aktivitas: membuat aplikasi penghitung dengan tombol tambah, kurang, dan reset.

Dari layar statis ke penghitung yang bereaksi



Wireframe sesi · penghitung Stateless vs Stateful

Peserta membangun layar kiri lebih dulu, merasakan bahwa angkanya tidak berubah, lalu mengubahnya menjadi layar kanan.

Tutorial langkah demi langkah

Langkah 1 — Kosongkan main.dart. Di proyek `latihan_satu`, hapus seluruh isi `lib/main.dart` dan mulai dari nol agar peserta melihat setiap widget yang dipakai.

Langkah 2 — Kerangka aplikasi dengan StatelessWidget.

```
import 'package:flutter/material.dart';

void main() => runApp(const AplikasiKu());

class AplikasiKu extends StatelessWidget {
  const AplikasiKu({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Penghitung',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal, useMaterial3: true),
      home: const HalamanPenghitung(),
    );
  }
}
```

Jelaskan: `MaterialApp` menyediakan tema dan navigasi; `home` adalah halaman pertama.

Langkah 3 — Halaman statis dulu. Buat versi Stateless dan jalankan:

```
class HalamanPenghitung extends StatelessWidget {
  const HalamanPenghitung({super.key});

  @override
  Widget build(BuildContext context) {
    int angka = 0;
    return Scaffold(
      appBar: AppBar(title: const Text('Penghitung')),
      body: Center(
        child: Text('$angka', style: const TextStyle(fontSize: 64)),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () { angka++; print(angka); },
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Tekan tombol beberapa kali. Angka di konsol bertambah, tetapi layar tetap 0. Ajak peserta menebak alasannya sebelum lanjut.

Langkah 4 — Ubah menjadi StatefulWidget. Di VS Code, letakkan kursor pada `StatelessWidget` → Ctrl+. → *Convert to StatefulWidget*. Pindahkan `angka` menjadi atribut kelas State dan bungkus perubahan dengan `setState`:

```
class HalamanPenghitung extends StatefulWidget {
  const HalamanPenghitung({super.key});

  @override
  State<HalamanPenghitung> createState() => _HalamanPenghitungState();
}

class _HalamanPenghitungState extends State<HalamanPenghitung> {
  int angka = 0;

  void tambah() => setState(() => angka++);
  void kurang() => setState(() { if (angka > 0) angka--; });
  void reset() => setState(() => angka = 0);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Penghitung')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text('$angka', style: const TextStyle(fontSize: 64)),
            const SizedBox(height: 24),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                ElevatedButton(onPressed: kurang, child: const Text('−')),
                const SizedBox(width: 12),
                OutlinedButton(onPressed: reset, child: const Text('Reset')),
                const SizedBox(width: 12),
                ElevatedButton(onPressed: tambah, child: const Text('+')),
              ],
            ),
          ],
        ),
```

```
      ),
    );
  }
}
```

Langkah 5 — Amati siklus hidup. Tambahkan di kelas State, lalu lakukan hot restart dan lihat urutan cetakan di konsol:

```
@override
void initState() { super.initState(); print('initState'); }

@override
void dispose() { print('dispose'); super.dispose(); }
```

Tambahkan juga `print('build');` di awal `build()` untuk menunjukkan bahwa setiap `setState` memicu `build` ulang.

Langkah 6 — Ekstrak widget. Pindahkan tampilan angka ke widget Stateless tersendiri:

```
class TampilanAngka extends StatelessWidget {
  final int nilai;
  const TampilanAngka({super.key, required this.nilai});

  @override
  Widget build(BuildContext context) {
    return Text('$nilai',
        style: TextStyle(
            fontSize: 64,
            color: nilai >= 10 ? Colors.red : Colors.black87));
  }
}
```

Ganti `Text('$angka', ...)` menjadi `TampilanAngka(nilai: angka)`. Tekankan pola: state disimpan di atas, widget di bawah cukup menerima data.

Langkah 7 — Lihat widget tree di DevTools. Saat aplikasi berjalan, buka DevTools: di VS Code tekan Ctrl+Shift+P → *Flutter: Open DevTools*, atau klik tautan DevTools yang muncul di terminal `flutter run`. Lakukan tiga hal bersama peserta:

1. Buka tab **Flutter Inspector**, aktifkan mode pilih widget, lalu ketuk angka di layar emulator. Inspector menyorot `TampilanAngka` di dalam pohon widget.
2. Telusuri pohon dari `MaterialApp` → `Scaffold` → `Column` → `Row`, dan cocokkan dengan kode yang ditulis.

3. Pilih `Row` tombol dan lihat panel properti/layout-nya untuk memahami bagaimana ruang dibagi antaranak.

Kebiasaan membuka Inspector ini akan sangat membantu saat menghadapi error layout di Hari 2.

Langkah 8 — Tantangan mandiri. Tambahkan pilihan langkah (1, 5, 10) memakai tiga `ChoiceChip`, sehingga tombol + dan – menambah atau mengurangi sesuai langkah yang dipilih.

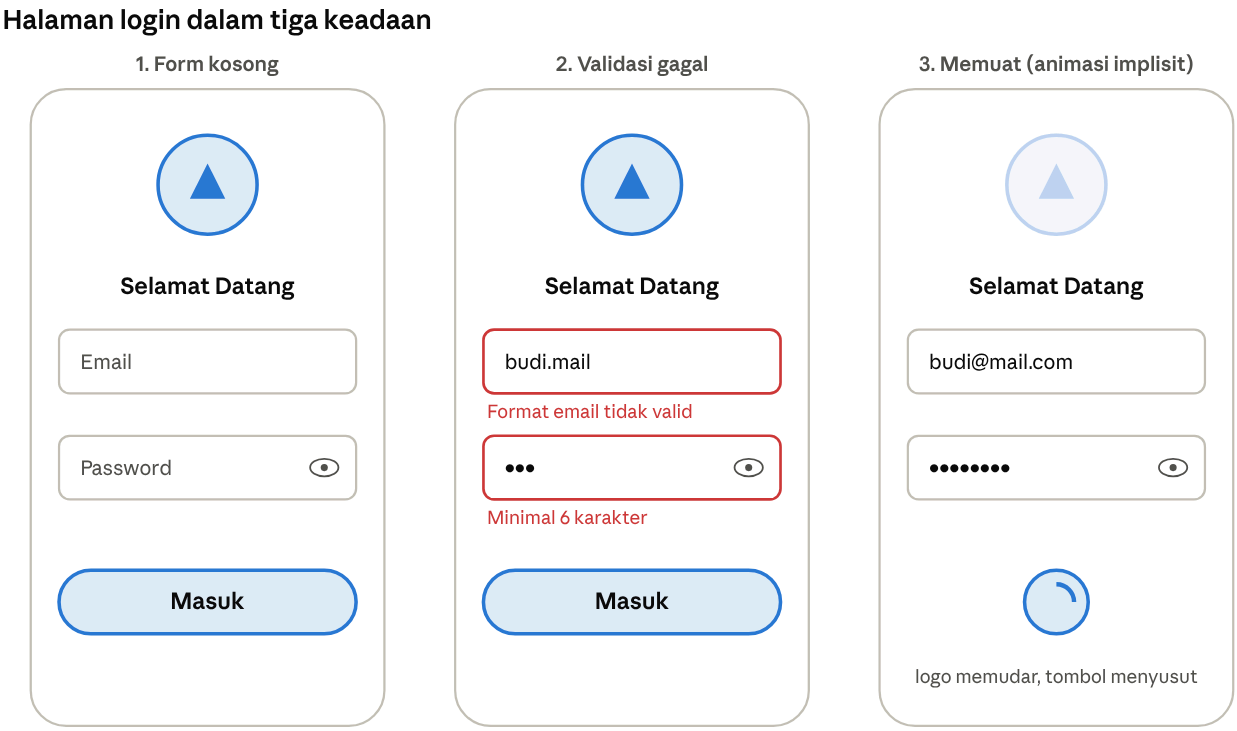
Checkpoint HOTS

1. (C4) Pada halaman profil berisi foto, nama, dan tombol "Ikuti" yang berubah menjadi "Mengikuti", analisis bagian mana yang cukup Stateless dan bagian mana yang harus Stateful.
2. (C4) Jelaskan apa yang terjadi pada widget tree ketika `setState()` dipanggil. Mengapa memanggilnya di widget paling atas bisa menurunkan performa?
3. (C5) Seorang peserta menjadikan semua widget Stateful "supaya aman". Nilailah keputusan ini dari sisi keterbacaan kode dan performa.
4. (C6) Rancang widget tree (dalam bentuk sketsa bertingkat) untuk kartu produk e-commerce: gambar, nama, harga, rating, dan tombol favorit.

Praktik: Membuat UI Sederhana (Halaman Login / Profil)

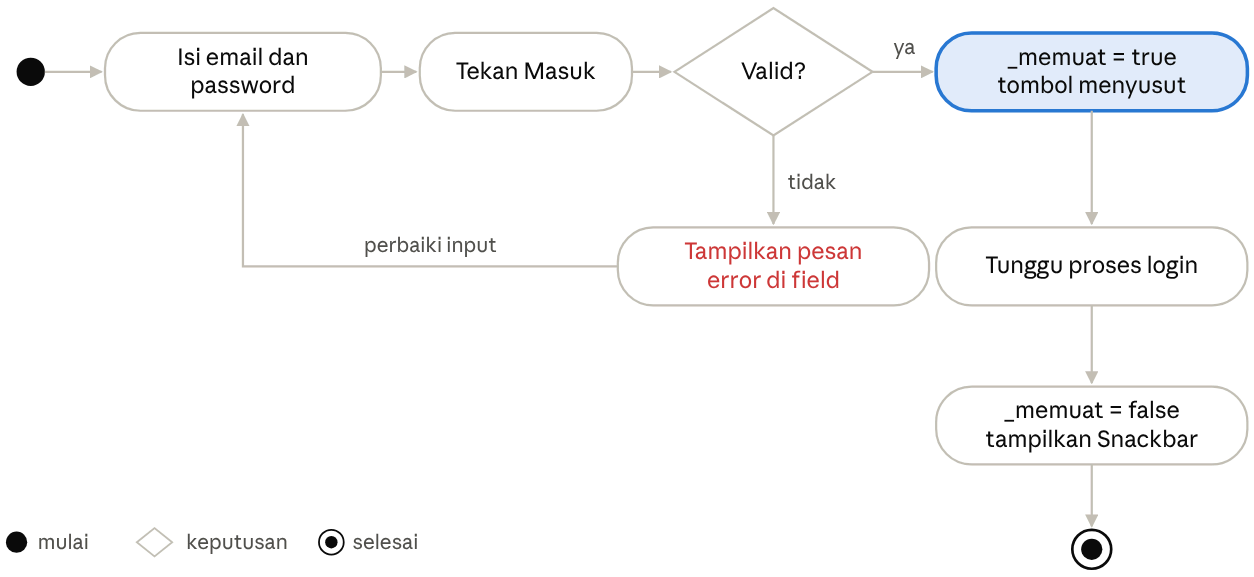
Tugas: buat halaman login dengan logo, field email dan password (bisa disembunyikan/ditampilkan), tombol "Masuk", dan validasi sederhana; atau halaman profil dengan foto, biodata, dan daftar keahlian.

Kriteria selesai: tampilan rapi di layar HP, tidak ada *overflow* kuning-hitam, dan kode dipecah menjadi minimal 2 widget kustom.



Wireframe praktik · login dalam tiga keadaan

Ketiga layar ini adalah satu halaman yang sama; yang berubah hanya state-nya. Gunakan gambar ini sebagai kriteria selesai saat berkeliling memeriksa peserta.



Konsep activity diagram · alur login

Activity diagram membantu peserta merancang alur sebelum menulis kode: setiap kotak menjadi satu baris logika di `_masuk()`, dan setiap belah ketupat menjadi satu `if`.

Tutorial langkah demi langkah (halaman login)

Langkah 1 — Buat proyek dan file baru. Jalankan `flutter create login_app`. Buat file `lib/halaman_login.dart`, lalu di `main.dart` set `home: const HalamanLogin()` (pakai kerangka `MaterialApp` dari Sesi 3).

Langkah 2 — Kerangka Stateful dengan Form. State dibutuhkan untuk menyimpan status tampil/sembunyi password.

```
import 'package:flutter/material.dart';

class HalamanLogin extends StatefulWidget {
  const HalamanLogin({super.key});
  @override
  State<HalamanLogin> createState() => _HalamanLoginState();
}

class _HalamanLoginState extends State<HalamanLogin> {
  final _formKey = GlobalKey<FormState>();
  final _emailC = TextEditingController();
  final _passC = TextEditingController();
  bool _sembunyi = true;

  @override
  void dispose() {
    _emailC.dispose();
    _passC.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(   // mencegah overflow saat keyboard
muncul
            padding: const EdgeInsets.all(24),
            child: Form(key: _formKey, child: _isiForm()),
          ),
```

```
        ),
      ),
    );
  }
}
```

Langkah 3 — Logo dan judul. Tambahkan method `_isiForm()` di dalam kelas State:

```
Widget _isiForm() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      const Icon(Icons.flutter_dash, size: 96, color: Colors.teal),
      const SizedBox(height: 16),
      const Text('Selamat Datang',
          textAlign: TextAlign.center,
          style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
      const SizedBox(height: 32),
      _fieldEmail(),
      const SizedBox(height: 16),
      _fieldPassword(),
      const SizedBox(height: 24),
      FilledButton(onPressed: _masuk, child: const Text('Masuk')),
    ],
  );
}
```

Langkah 4 — Field email dengan validasi.

```
Widget _fieldEmail() => TextFormField(
      controller: _emailC,
      keyboardType: TextInputType.emailAddress,
      decoration: const InputDecoration(
        labelText: 'Email',
        prefixIcon: Icon(Icons.email_outlined),
        border: OutlineInputBorder(),
      ),
      validator: (v) {
        if (v == null || v.isEmpty) return 'Email wajib diisi';
        if (!v.contains('@')) return 'Format email tidak valid';
        return null;
      },
    );
```

Langkah 5 — Field password dengan tombol mata.

```
Widget _fieldPassword() => TextFormField(
      controller: _passC,
      obscureText: _sembunyi,
      decoration: InputDecoration(
        labelText: 'Password',
        prefixIcon: const Icon(Icons.lock_outline),
        border: const OutlineInputBorder(),
        suffixIcon: IconButton(
          icon: Icon(_sembunyi ? Icons.visibility : Icons.visibility_off),
          onPressed: () => setState(() => _sembunyi = !_sembunyi),
        ),
      ),
      validator: (v) =>
          (v == null || v.length < 6) ? 'Minimal 6 karakter' : null,
    );
```

Langkah 6 — Aksi tombol Masuk.

```
void _masuk() {
  if (!_formKey.currentState!.validate()) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Login sebagai ${_emailC.text}')),
  );
}
```

Langkah 7 — Uji. Coba kirim form kosong, email tanpa @, dan password 3 karakter. Lalu ketuk field password di HP/emulator: pastikan tidak muncul garis kuning-hitam saat keyboard tampil. Coba hapus `SingleChildScrollView` untuk melihat overflow, lalu kembalikan.

Langkah 8 — Rapikan menjadi widget kustom. Pindahkan `_fieldEmail` dan `_fieldPassword` menjadi kelas widget terpisah (misalnya `FieldEmail` yang menerima `controller`) di folder `lib/widgets/`, sesuai kriteria minimal 2 widget kustom.

Langkah 9 — Tambahkan animasi implisit. Animasi implisit cukup dengan mengganti nilai properti; widget `Animated...` mengurus transisinya. Tambahkan status memuat di kelas State:

```
bool _memuat = false;

Future<void> _masuk() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() => _memuat = true);
  await Future.delayed(const Duration(seconds: 2)); // simulasi proses login
  if (!mounted) return;
  setState(() => _memuat = false);
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Login sebagai ${_emailC.text}')),
  );
}
```

Lalu di `_isiForm()`, bungkus logo dengan `AnimatedOpacity` dan ganti `FilledButton` dengan tombol yang menyusut menjadi lingkaran saat memuat:

```
AnimatedOpacity(
  opacity: _memuat ? 0.3 : 1.0,
  duration: const Duration(milliseconds: 300),
  child: const Icon(Icons.flutter_dash, size: 96, color: Colors.teal),
),

// ... field email dan password ...

Center(
  child: AnimatedContainer(
    duration: const Duration(milliseconds: 300),
    curve: Curves.easeInOut,
    width: _memuat ? 48 : 280,
    height: 48,
    child: _memuat
        ? const CircularProgressIndicator()
        : FilledButton(onPressed: _masuk, child: const Text('Masuk')),
  ),
),
```

`Center` diperlukan karena `Column` ber-`stretch` memaksa lebar anaknya; tanpa `Center`, properti `width` diabaikan. Coba ubah `curve` menjadi `Curves.bounceOut` dan `duration` menjadi 1 detik untuk merasakan bedanya.

Varian halaman profil: gunakan `CircleAvatar` (foto), `Text` (nama dan jabatan), `Card` + `ListTile` (biodata), dan `Wrap` + `Chip` (daftar keahlian), dibungkus `SingleChildScrollView`.

Checkpoint HOTS

1. (C4) Saat keyboard muncul, form login Anda mengalami overflow. Analisis penyebabnya dan bandingkan dua cara mengatasinya.
2. (C5) Tukar aplikasi dengan teman sebelah. Evaluasi halaman login mereka dari sisi kemudahan pakai (UX): sebutkan satu kekuatan dan satu perbaikan beserta alasannya.
3. (C6) Kembangkan halaman login Anda agar tetap nyaman dipakai di tablet maupun HP. Perubahan layout apa yang Anda rancang?

Hari 2: Layout Lanjutan, Navigasi, & API

Hari kedua membawa peserta dari satu halaman statis ke aplikasi multi-halaman yang menampilkan data nyata dari internet. Urutan sesinya mengikuti Flutter Learning Pathway: ambil data dulu, lalu bangun tampilan yang kaya dan adaptif, terakhir hubungkan halaman lewat navigasi.

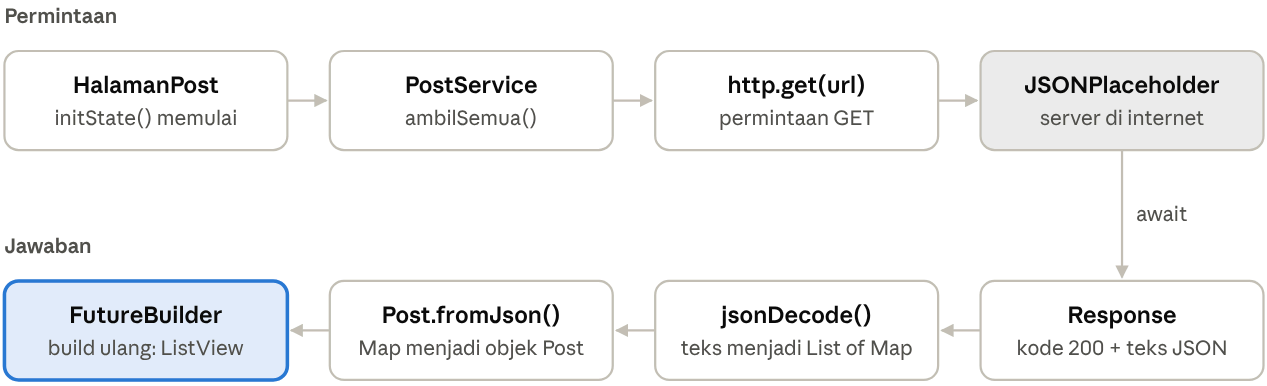
Sesi 1: Melakukan HTTP Request

Tujuan: peserta mampu mengambil data JSON dari API publik dan menampilkannya.

Materi pokok

- Konsep REST API, metode GET/POST, kode status (200, 404, 500), dan format JSON.
- Pemrograman asinkron: `Future`, `async` / `await`, penanganan error dengan `try` / `catch`.
- Paket `http`: menambah dependensi di `pubspec.yaml`, mengirim GET request.
- Parsing JSON menjadi kelas model (`fromJson`).
- `FutureBuilder` dan tiga keadaan tampilan: memuat, berhasil, gagal.
- Contoh API latihan: JSONPlaceholder, PokeAPI, atau API berita publik.

Data berjalan dari server ke layar melalui delapan tahap



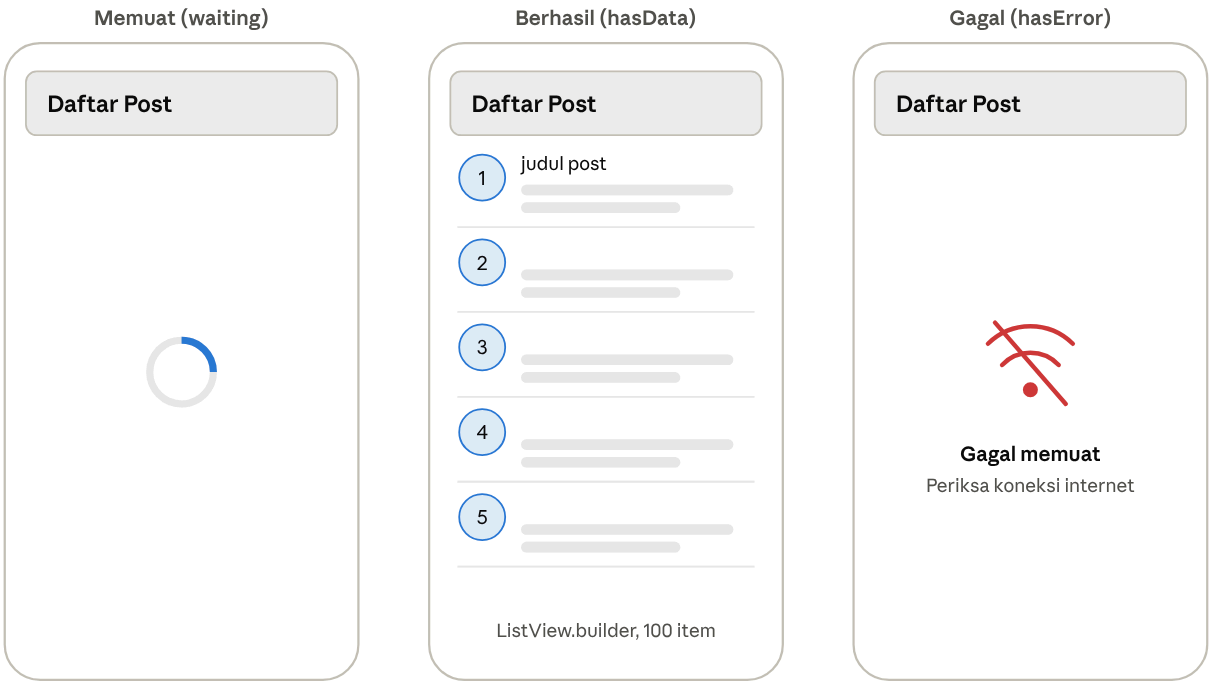
Error bisa muncul di tiap tahap: tanpa internet, kode 404/500, atau format JSON berbeda. Semuanya berakhir di cabang snap.hasError pada FutureBuilder.

Konsep alur data · HTTP request sampai layar

Gunakan diagram ini saat menjawab HOTS nomor 1: setiap kotak adalah titik di mana error bisa terjadi dan perlu ditangani.

Aktivitas: mengambil daftar posting dari JSONPlaceholder dan menampilkannya dalam `ListView`.

FutureBuilder menampilkan satu dari tiga keadaan



Wireframe sesi · tiga keadaan FutureBuilder

Pada Langkah 7, minta peserta memunculkan ketiga layar ini secara sengaja: jalankan normal, lalu aktifkan mode pesawat.

Tutorial langkah demi langkah

Langkah 1 — Lihat data mentahnya. Buka jsonplaceholder.typicode.com/posts di browser. Tunjukkan bahwa respons berupa *array* JSON berisi objek dengan kunci `userId`, `id`, `title`, dan `body`.

Langkah 2 — Tambah paket http. Di folder proyek (boleh proyek baru `berita_app`) jalankan:

```
flutter pub add http
```

Periksa `pubspec.yaml`: baris `http:` muncul di bagian `dependencies`.

Langkah 3 — Izin internet Android. Buka `android/app/src/main/AndroidManifest.xml` dan pastikan baris ini ada tepat di atas tag `<application`:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Tanpa baris ini, aplikasi mode debug tetap bisa mengakses internet, tetapi .apk rilis tidak. Ini sumber bug klasik di Hari 3.

Langkah 4 — Buat kelas model. `lib/models/post.dart`:

```
class Post {
  final int id;
  final String judul;
  final String isi;

  Post({required this.id, required this.judul, required this.isi});

  factory Post.fromJson(Map<String, dynamic> json) => Post(
        id: json['id'] as int,
        judul: json['title'] as String,
        isi: json['body'] as String,
      );
}
```

Langkah 5 — Buat service untuk memanggil API. `lib/services/post_service.dart`:

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/post.dart';

class PostService {
  static const _url = 'https://jsonplaceholder.typicode.com/posts';

  Future<List<Post>> ambilSemua() async {
    final res = await http
        .get(Uri.parse(_url))
        .timeout(const Duration(seconds: 10));
    if (res.statusCode != 200) {
      throw Exception('Server mengembalikan kode ${res.statusCode}');
    }
    final List data = jsonDecode(res.body);
    return data.map((e) => Post.fromJson(e)).toList();
  }
}
```

Jelaskan alur `async` / `await`: fungsi "menunggu" tanpa membekukan layar.

Langkah 6 — Tampilkan dengan FutureBuilder. `lib/pages/halaman_post.dart`:

```
class HalamanPost extends StatefulWidget {
  const HalamanPost({super.key});
  @override
  State<HalamanPost> createState() => _HalamanPostState();
}

class _HalamanPostState extends State<HalamanPost> {
  late Future<List<Post>> _future;

  @override
  void initState() {
    super.initState();
    _future = PostService().ambilSemua();   // dipanggil sekali, bukan di
build()
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Post')),
```

```
      body: FutureBuilder<List<Post>>(
        future: _future,
        builder: (context, snap) {
          if (snap.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snap.hasError) {
            return Center(child: Text('Gagal memuat: ${snap.error}'));
          }
          final posts = snap.data!;
          if (posts.isEmpty) return const Center(child: Text('Belum ada
data'));
          return ListView.builder(
            itemCount: posts.length,
            itemBuilder: (_, i) => ListTile(
              leading: CircleAvatar(child: Text('${posts[i].id}')),
              title: Text(posts[i].judul, maxLines: 1, overflow:
TextOverflow.ellipsis),
              subtitle: Text(posts[i].isi, maxLines: 2, overflow:
TextOverflow.ellipsis),
            ),
          );
        },
      ),
    );
  }
}
```

Langkah 7 — Uji tiga keadaan. Jalankan dan amati indikator loading lalu daftar. Kemudian aktifkan mode pesawat di emulator/HP dan hot restart: pesan gagal harus muncul, bukan crash. Terakhir, ubah URL menjadi `/postsxx` untuk memicu kode 404.

Langkah 8 — Tunjukkan jebakan umum. Pindahkan `PostService().ambilSemua()` langsung ke properti `future:` di dalam `build()`, tambahkan tombol yang memanggil `setState`, lalu tunjukkan bahwa API dipanggil ulang setiap kali layar di-build. Kembalikan ke `initState`.

Langkah 9 — Tantangan mandiri. Ganti sumber data ke PokeAPI (`https://pokeapi.co/api/v2/pokemon?limit=30`) dan tampilkan nama Pokémon. Perhatikan bahwa daftarnya ada di dalam kunci `results`, bukan array langsung.

Checkpoint HOTS

1. (C4) Uraikan alur data dari server sampai tampil di layar: request, response, decode JSON, model, widget. Di titik mana saja error bisa terjadi?
2. (C4) Mengapa aplikasi tetap responsif saat menunggu data, padahal Dart berjalan di satu *thread*? Kaitkan dengan konsep `Future` dan *event loop*.
3. (C5) Seorang peserta langsung memakai `Map` hasil `jsonDecode` di widget tanpa membuat kelas model. Nilailah pendekatan ini untuk proyek yang akan terus dikembangkan.
4. (C5) Saat sinyal hilang, aplikasi hanya menampilkan layar putih. Evaluasi pengalaman pengguna ini dan usulkan perbaikan tampilan untuk setiap keadaan (memuat, kosong, gagal).
5. (C6) Rancang strategi agar daftar berita tetap tampil saat offline, menggunakan data terakhir yang berhasil diambil.

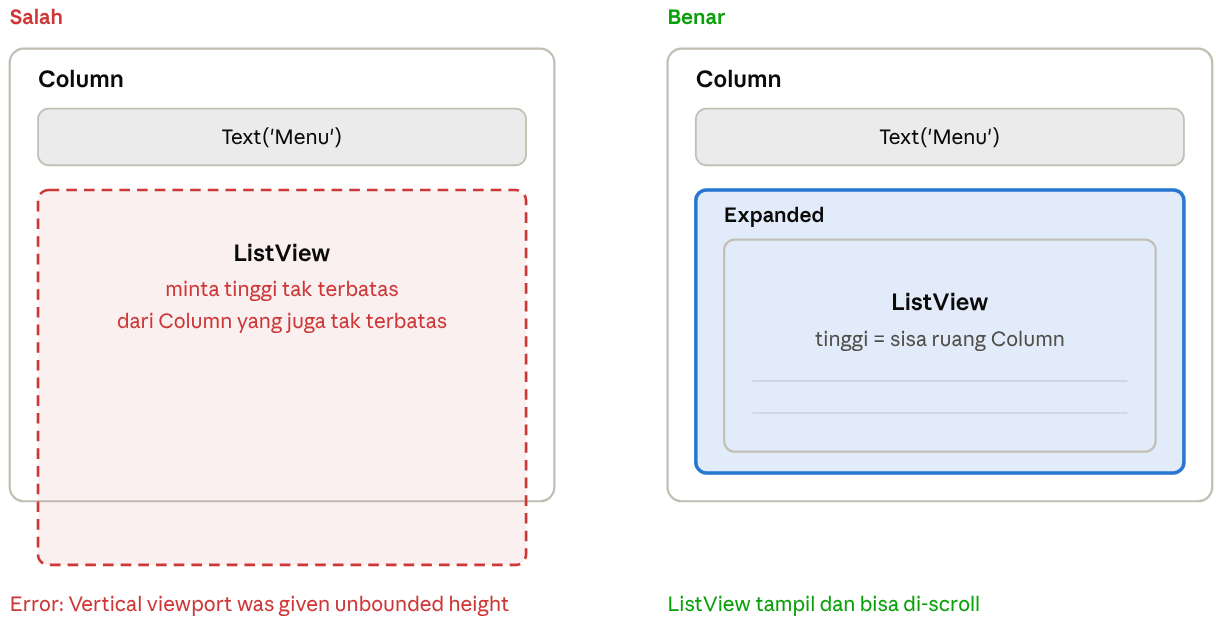
Sesi 2: Membangun Layout Kompleks

Tujuan: peserta mampu menampilkan kumpulan data dalam bentuk daftar, grid, dan scroll gabungan.

Materi pokok

- `ListView`, `ListView.builder`, dan `ListView.separated`; kapan memakai masing-masing.
- `GridView.count` dan `GridView.builder` untuk katalog.
- `CustomScrollView` dengan `SliverAppBar`, `SliverList`, dan `SliverGrid`.
- `Expanded`, `Flexible`, `Stack`, dan `MediaQuery` / `LayoutBuilder` untuk layout responsif.

ListView di dalam Column butuh Expanded agar punya batas tinggi

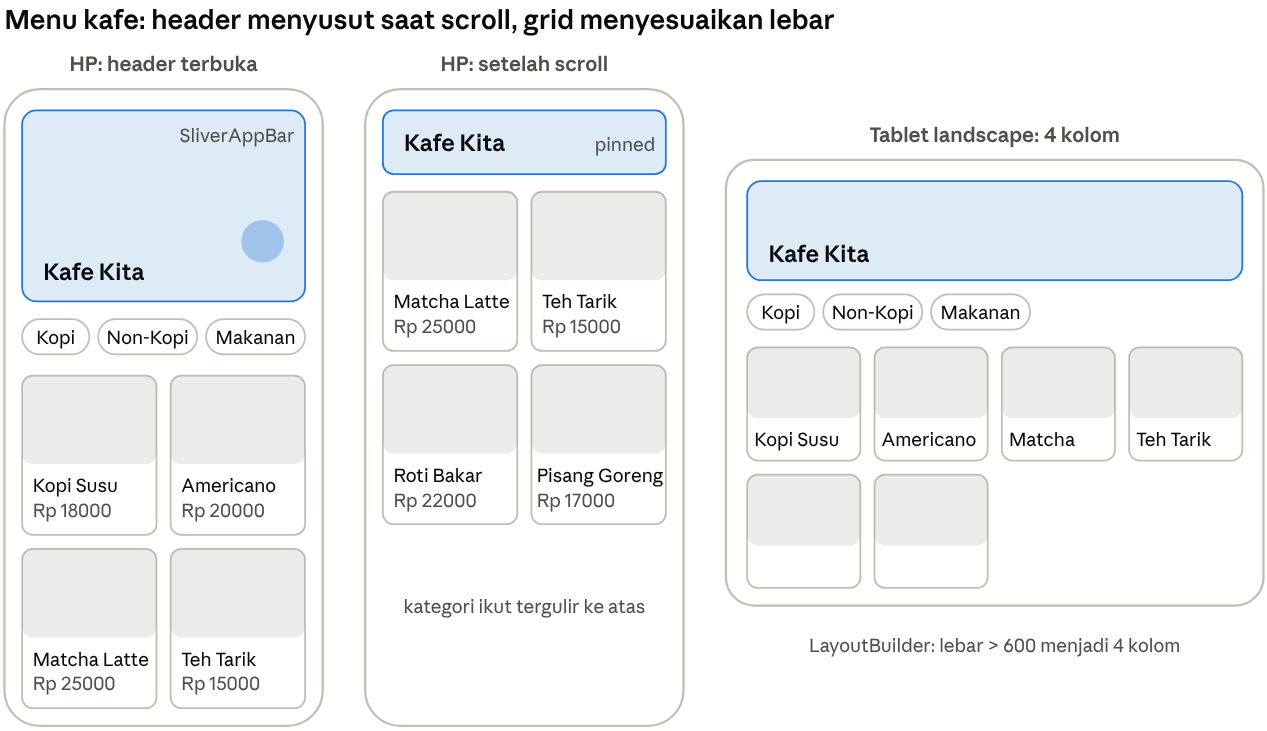


Aturan layout Flutter: batasan turun dari induk, ukuran naik dari anak, induk menentukan posisi.

Konsep layout · constraints dan Expanded

Gambar ini adalah jawaban visual untuk Langkah 6 dan HOTS nomor 2: `Expanded` mengubah tinggi tak terbatas menjadi batas yang jelas.

Aktivitas: membuat halaman menu kafe: header gambar yang mengecil saat di-scroll, kategori horizontal, dan grid menu.



Wireframe sesi · menu kafe di HP dan tablet

Ketiga tampilan berasal dari satu kode `CustomScrollView` yang sama; hanya lebar layar dan posisi scroll yang berbeda.

Tutorial langkah demi langkah

Proyek `kafe_app` ini dipakai terus sampai Hari 3, jadi minta peserta menyimpannya baik-baik.

Langkah 1 — Buat proyek dan model data. Jalankan `flutter create kafe_app`, lalu buat `lib/models/menu.dart`:

```
class Menu {
  final int id;
  final String nama;
  final String kategori;
  final int harga;
  final String gambar;

  const Menu(this.id, this.nama, this.kategori, this.harga, this.gambar);
}

const daftarMenu = [
  Menu(1, 'Kopi Susu', 'Kopi', 18000, 'https://picsum.photos/seed/kopi/400'),
  Menu(2, 'Americano', 'Kopi', 20000, 'https://picsum.photos/seed/ame/400'),
  Menu(3, 'Matcha Latte', 'Non-Kopi', 25000,
'https://picsum.photos/seed/matcha/400'),
  Menu(4, 'Teh Tarik', 'Non-Kopi', 15000,
'https://picsum.photos/seed/teh/400'),
  Menu(5, 'Roti Bakar', 'Makanan', 22000,
'https://picsum.photos/seed/roti/400'),
  Menu(6, 'Pisang Goreng', 'Makanan', 17000,
'https://picsum.photos/seed/pisang/400'),
];
```

Langkah 2 — Daftar sederhana dengan ListView.builder. Buat `lib/pages/halaman_menu.dart` versi awal untuk memahami builder:

```
ListView.builder(
  itemCount: daftarMenu.length,
  itemBuilder: (context, i) {
    final m = daftarMenu[i];
    return ListTile(
      leading: CircleAvatar(backgroundImage: NetworkImage(m.gambar)),
      title: Text(m.nama),
      subtitle: Text('Rp ${m.harga}'),
    );
  },
)
```

Jalankan, lalu ganti dengan `ListView.separated` (tambahkan `separatorBuilder: (_, __) => const Divider()`) dan bandingkan hasilnya.

Langkah 3 — Kartu menu sebagai widget sendiri. Buat `lib/widgets/kartu_menu.dart`:

```
class KartuMenu extends StatelessWidget {
  final Menu menu;
  final VoidCallback? onTap;
  const KartuMenu({super.key, required this.menu, this.onTap});

  @override
  Widget build(BuildContext context) {
    return Card(
      clipBehavior: Clip.antiAlias,
      child: InkWell(
        onTap: onTap,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Expanded(child: Image.network(menu.gambar, fit: BoxFit.cover,
width: double.infinity)),
            Padding(
              padding: const EdgeInsets.all(8),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(menu.nama, style: const TextStyle(fontWeight:
FontWeight.bold)),
                  Text('Rp ${menu.harga}'),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Langkah 4 — Halaman dengan CustomScrollView. Ganti isi halaman menu:

```
class HalamanMenu extends StatelessWidget {
  const HalamanMenu({super.key});

  @override
  Widget build(BuildContext context) {
    final kategori = daftarMenu.map((m) => m.kategori).toSet().toList();
    final lebar = MediaQuery.of(context).size.width;
```

```
    final kolom = lebar > 600 ? 4 : 2;      // responsif: HP 2, tablet 4

    return Scaffold(
      body: CustomScrollView(
        slivers: [
          SliverAppBar(
            expandedHeight: 200,
            pinned: true,
            flexibleSpace: FlexibleSpaceBar(
              title: const Text('Kafe Kita'),
              background:
Image.network('https://picsum.photos/seed/kafe/800', fit: BoxFit.cover),
            ),
          ),
          SliverToBoxAdapter(
            child: SizedBox(
              height: 56,
              child: ListView.separated(
                scrollDirection: Axis.horizontal,
                padding: const EdgeInsets.all(8),
                itemCount: kategori.length,
                separatorBuilder: (_, __) => const SizedBox(width: 8),
                itemBuilder: (_, i) => Chip(label: Text(kategori[i])),
              ),
            ),
          ),
          SliverPadding(
            padding: const EdgeInsets.all(8),
            sliver: SliverGrid.builder(
              gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
                crossAxisCount: kolom,
                childAspectRatio: 0.8,
                crossAxisSpacing: 8,
                mainAxisSpacing: 8,
              ),
              itemCount: daftarMenu.length,
              itemBuilder: (_, i) => KartuMenu(menu: daftarMenu[i]),
            ),
          ),
        ],
```

```
      ),
    );
  }
}
```

Langkah 5 — Uji perilaku scroll. Scroll ke atas dan bawah: header mengecil dan tetap terlihat karena `pinned: true`. Ubah menjadi `floating: true, pinned: false` dan bandingkan. Putar emulator ke mode landscape untuk melihat jumlah kolom berubah.

Langkah 6 — Demonstrasi error constraints. Tunjukkan kesalahan umum: taruh `ListView` langsung di dalam `Column` tanpa `Expanded`, jalankan, baca pesan "unbounded height", lalu perbaiki dengan membungkusnya `Expanded`.

Langkah 7 — Tantangan mandiri. Buat chip kategori bisa dipilih (`ChoiceChip` + StatefulWidget) sehingga grid hanya menampilkan menu dari kategori terpilih.

Checkpoint HOTS

1. (C4) Bandingkan `ListView(children: [...])` dengan `ListView.builder` untuk menampilkan 10.000 item. Mana yang lebih hemat memori dan mengapa?
2. (C4) Anda menaruh `ListView` di dalam `Column` dan muncul error "unbounded height". Analisis penyebabnya dari cara Flutter mengukur ukuran widget (*constraints*).
3. (C5) Untuk halaman beranda marketplace (banner, kategori, produk terlaris, rekomendasi), evaluasi apakah lebih baik memakai beberapa ListView bersarang atau satu `CustomScrollView` dengan sliver.
4. (C6) Rancang layout galeri foto yang menampilkan 2 kolom di HP dan 4 kolom di tablet. Sebutkan widget dan logika yang Anda gunakan.

Sesi 3: Navigasi dan Routing Antarhalaman

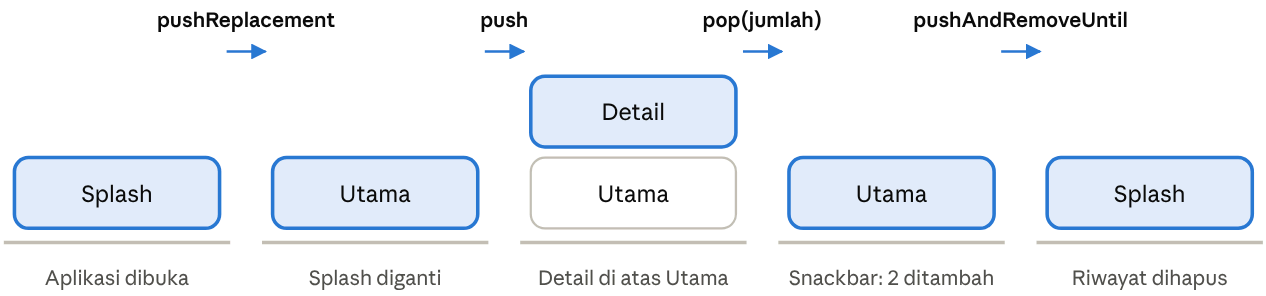
Tujuan: peserta mampu berpindah halaman sambil mengirim dan menerima data.

Materi pokok

- Konsep tumpukan halaman (*stack*): `Navigator.push`, `pop`, `pushReplacement`, `pushAndRemoveUntil`.
- Mengirim data lewat konstruktor dan menerima hasil balik dari `pop`.
- *Named routes* dan pengenalan paket `go_router`.
- `BottomNavigationBar` dan `TabBar` untuk navigasi utama.

Navigator bekerja seperti tumpukan halaman

Halaman paling atas yang tampil di layar



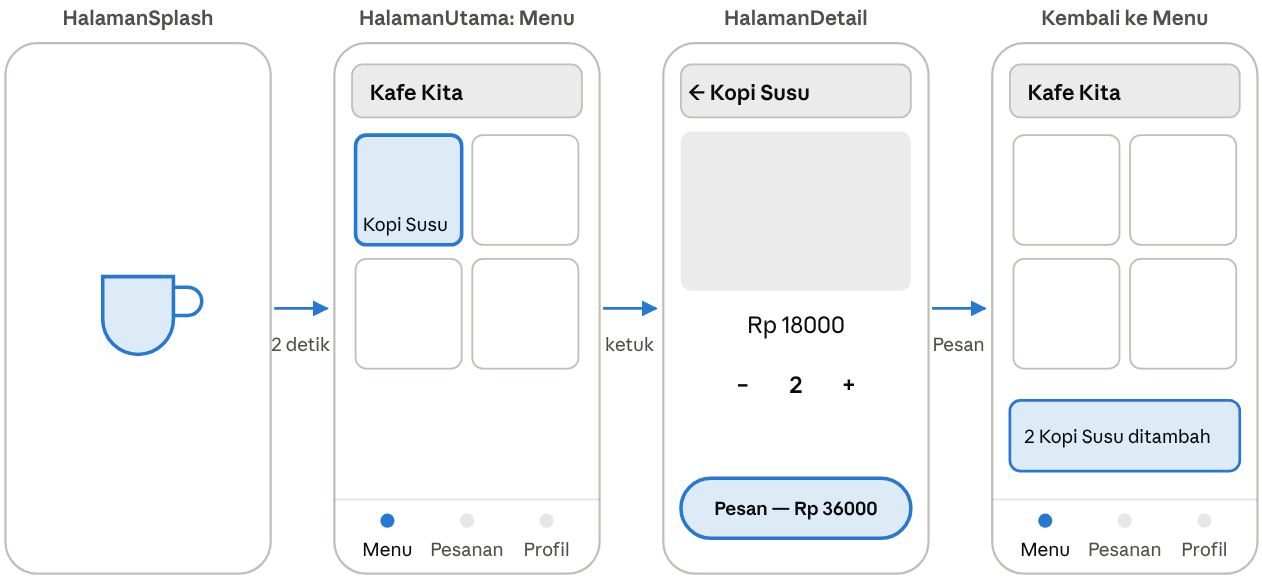
Tombol back sama dengan pop: halaman teratas dibuang, halaman di bawahnya tampil lagi.

Konsep navigasi · tumpukan halaman

Gambar tumpukan ini di papan setiap kali memanggil method Navigator; peserta jadi paham mengapa setelah `pushReplacement` tombol back langsung menutup aplikasi.

Aktivitas: dari grid menu kafe, ketuk item untuk membuka halaman detail, lalu kembali membawa jumlah pesanan.

Dari splash ke menu, buka detail, lalu kembali membawa jumlah



Wireframe sesi · alur navigasi kafe_app

Empat layar ini mewakili Langkah 2 sampai 4 tutorial; uji juga skenario tombol back di layar Detail, yang kembali ke Menu tanpa snackbar.

Tutorial langkah demi langkah

Langkah 1 — Buat halaman detail. Buat `lib/pages/halaman_detail.dart` yang menerima objek `Menu` lewat konstruktor:

```
class HalamanDetail extends StatefulWidget {
  final Menu menu;
  const HalamanDetail({super.key, required this.menu});
  @override
  State<HalamanDetail> createState() => _HalamanDetailState();
}

class _HalamanDetailState extends State<HalamanDetail> {
  int jumlah = 1;

  @override
  Widget build(BuildContext context) {
    final m = widget.menu;
    return Scaffold(
      appBar: AppBar(title: Text(m.nama)),
      body: Column(
        children: [
          Image.network(m.gambar, height: 250, width: double.infinity, fit:
BoxFit.cover),
          const SizedBox(height: 16),
          Text('Rp ${m.harga}', style: const TextStyle(fontSize: 22)),
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              IconButton(onPressed: () => setState(() { if (jumlah > 1)
jumlah--; }), icon: const Icon(Icons.remove)),
              Text('$jumlah', style: const TextStyle(fontSize: 22)),
              IconButton(onPressed: () => setState(() => jumlah++), icon:
const Icon(Icons.add)),
            ],
          ),
          const Spacer(),
          Padding(
            padding: const EdgeInsets.all(16),
            child: FilledButton(
              onPressed: () => Navigator.pop(context, jumlah),   // kirim
hasil balik
              child: Text('Pesan — Rp ${m.harga * jumlah}'),
```

```
            ),
          ),
        ],
      ),
    );
  }
}
```

Langkah 2 — Push dan tunggu hasil. Di `HalamanMenu`, isi `onTap` pada `KartuMenu`:

```
itemBuilder: (_, i) => KartuMenu(
  menu: daftarMenu[i],
  onTap: () async {
    final jumlah = await Navigator.push<int>(
      context,
      MaterialPageRoute(builder: (_) => HalamanDetail(menu: daftarMenu[i])),
    );
    if (jumlah != null && context.mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('$jumlah ${daftarMenu[i].nama} ditambahkan')),
      );
    }
  },
),
```

Uji dua skenario: tekan tombol Pesan (snackbar muncul) dan tekan tombol back (jumlah = null, tidak ada snackbar).

Langkah 3 — Halaman splash dengan pushReplacement. Buat `HalamanSplash` yang otomatis pindah ke menu setelah 2 detik:

```
class HalamanSplash extends StatefulWidget {
  const HalamanSplash({super.key});
  @override
  State<HalamanSplash> createState() => _HalamanSplashState();
}

class _HalamanSplashState extends State<HalamanSplash> {
  @override
  void initState() {
    super.initState();
    Future.delayed(const Duration(seconds: 2), () {
      if (!mounted) return;
```

```
      Navigator.pushReplacement(context,
          MaterialPageRoute(builder: (_) => const HalamanUtama()));
    });
  }

  @override
  Widget build(BuildContext context) =>
      const Scaffold(body: Center(child: Icon(Icons.local_cafe, size: 96)));
}
```

Set `home: const HalamanSplash()`. Setelah pindah ke menu, tekan back: aplikasi keluar, tidak kembali ke splash. Lalu ganti sementara dengan `Navigator.push` untuk menunjukkan bedanya.

Langkah 4 — Navigasi utama dengan NavigationBar. Buat `HalamanUtama` berisi tiga tab:

```
class HalamanUtama extends StatefulWidget {
  const HalamanUtama({super.key});
  @override
  State<HalamanUtama> createState() => _HalamanUtamaState();
}

class _HalamanUtamaState extends State<HalamanUtama> {
  int _indeks = 0;
  final _halaman = const [
    HalamanMenu(),
    Center(child: Text('Pesanan')),
    Center(child: Text('Profil')),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: IndexedStack(index: _indeks, children: _halaman),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _indeks,
        onDestinationSelected: (i) => setState(() => _indeks = i),
        destinations: const [
          NavigationDestination(icon: Icon(Icons.restaurant_menu), label:
'Menu'),
          NavigationDestination(icon: Icon(Icons.receipt_long), label:
'Pesanan'),
          NavigationDestination(icon: Icon(Icons.person), label: 'Profil'),
```

```
        ],
      ),
    );
  }
}
```

Jelaskan peran `IndexedStack`: posisi scroll tiap tab tetap terjaga saat berpindah tab.

Langkah 5 — Pengenalan named routes (demo fasilitator). Tunjukkan cara mendaftarkan rute di `MaterialApp`:

```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (_) => const HalamanSplash(),
    '/utama': (_) => const HalamanUtama(),
  },
)
// pemanggilan:
Navigator.pushReplacementNamed(context, '/utama');
```

Cukup disebutkan bahwa untuk aplikasi besar dan deep link, paket `go_router` lebih dianjurkan.

Langkah 6 — Tantangan mandiri. Tambahkan tombol "Logout" di tab Profil yang kembali ke splash memakai `Navigator.pushAndRemoveUntil` sehingga seluruh riwayat halaman terhapus.

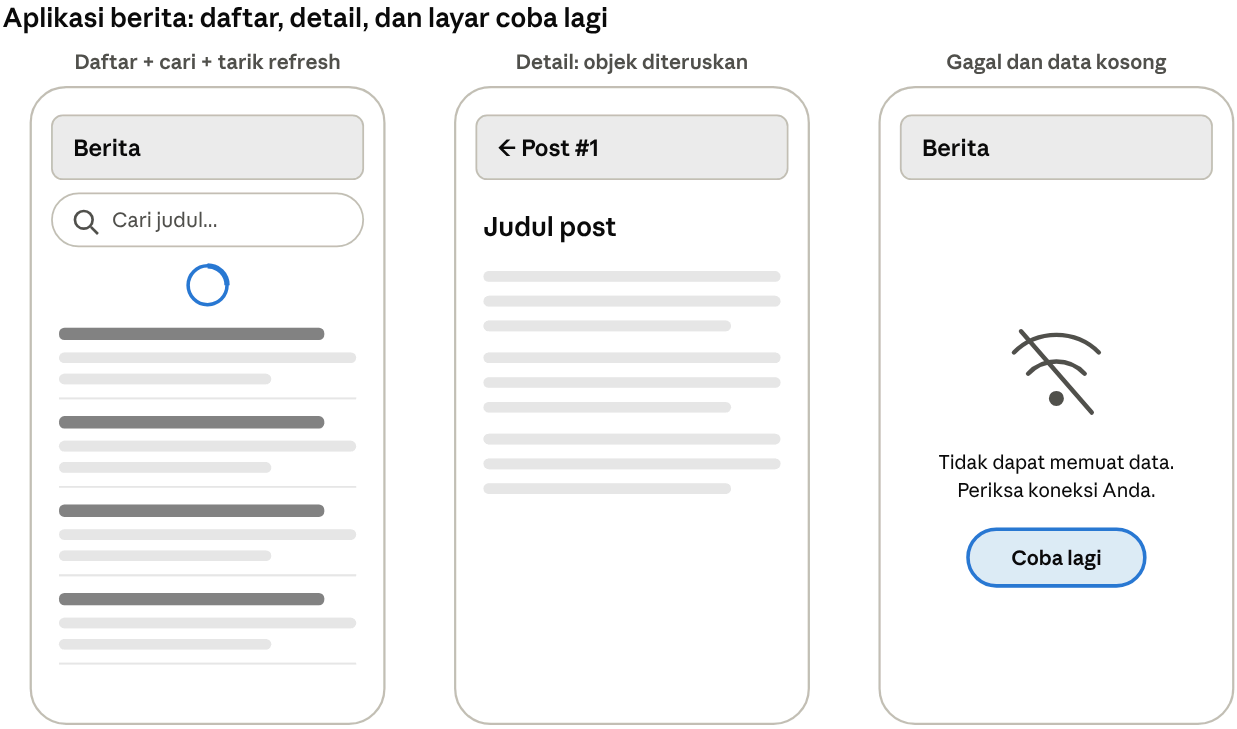
Checkpoint HOTS

1. (C4) Setelah login berhasil, pengguna menekan tombol back dan kembali ke halaman login. Analisis kesalahan navigasinya dan method mana yang seharusnya dipakai.
2. (C4) Bandingkan mengirim data lewat konstruktor dengan lewat *route arguments*. Apa dampaknya terhadap keamanan tipe data dan kemudahan perawatan kode?
3. (C5) Aplikasi Anda punya 25 halaman dan butuh *deep link* dari notifikasi. Evaluasi apakah `Navigator.push` biasa masih memadai atau perlu `go_router`.
4. (C6) Rancang peta navigasi (flow) aplikasi pemesanan tiket bioskop dari splash screen sampai e-tiket, termasuk halaman mana yang tidak boleh bisa di-back.

Praktik: Aplikasi Katalog Data atau Berita Sederhana

Tugas: buat aplikasi minimal 2 halaman: halaman daftar (data dari API) dan halaman detail. Tambahkan indikator loading, pesan error dengan tombol "Coba lagi", dan *pull-to-refresh*.

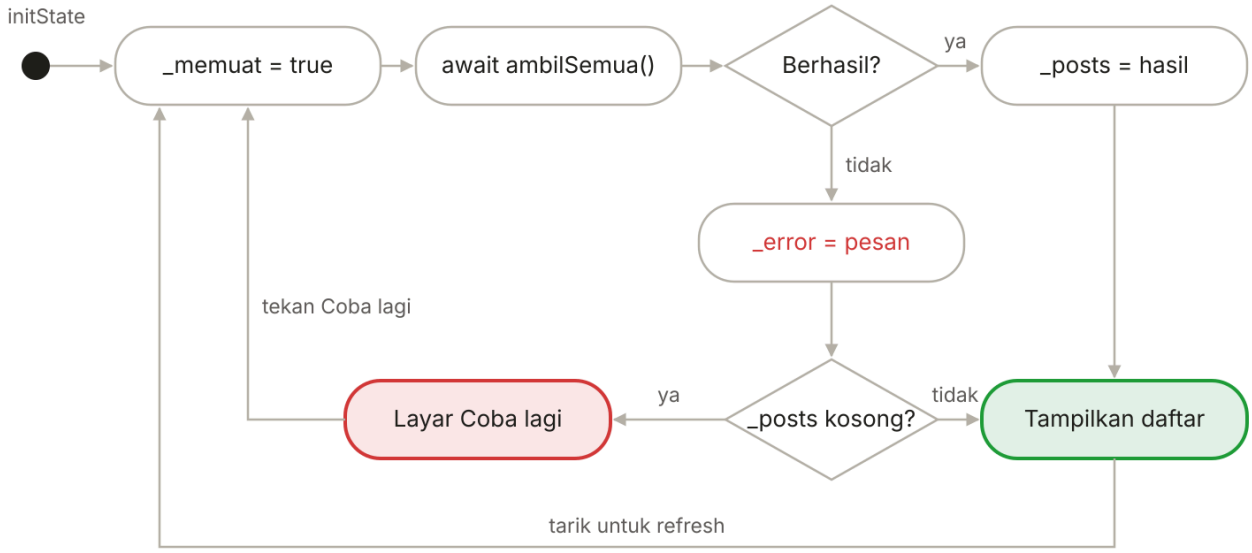
Kriteria selesai: data tampil dari API, navigasi ke detail membawa data yang benar, dan aplikasi tidak *crash* saat internet dimatikan.



Wireframe praktik · aplikasi berita

Ini gambaran minimal yang harus dicapai; fitur pencarian di layar kiri adalah contoh fitur orisinal dari Langkah 5.

Setiap pemuatan berakhir di daftar atau di layar Coba lagi



Blok finally menyetel _memuat = false di kedua cabang. Data lama tetap tampil bila refresh gagal.

Diagram ini memetakan kode Langkah 1 dan 2: tiga pintu masuk (initState, tarik refresh, Coba lagi) semuanya memanggil `_muat()` yang sama.

Tutorial langkah demi langkah

Tutorial ini melanjutkan `berita_app` dari Sesi 3. Peserta boleh memakai API lain (PokeAPI, fakestoreapi.com untuk katalog produk) dengan pola yang sama.

Langkah 1 — Ganti FutureBuilder dengan state manual. Agar tombol "Coba lagi" dan pull-to-refresh mudah dibuat, simpan keadaan di variabel sendiri:

```
class _HalamanPostState extends State<HalamanPost> {
  final _service = PostService();
  List<Post> _posts = [];
  bool _memuat = true;
  String? _error;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  Future<void> _muat() async {
    setState(() { _memuat = true; _error = null; });
```

```
    try {
      final hasil = await _service.ambilSemua();
      setState(() => _posts = hasil);
    } catch (e) {
      setState(() => _error = 'Tidak dapat memuat data. Periksa koneksi
Anda.');
    } finally {
      if (mounted) setState(() => _memuat = false);
    }
  }
```

Langkah 2 — Tampilan untuk setiap keadaan. Lanjutkan kelas yang sama:

```
  Widget _isi() {
    if (_memuat && _posts.isEmpty) {
      return const Center(child: CircularProgressIndicator());
    }
    if (_error != null && _posts.isEmpty) {
      return Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Icon(Icons.wifi_off, size: 64),
            const SizedBox(height: 8),
            Text(_error!),
            const SizedBox(height: 8),
            FilledButton(onPressed: _muat, child: const Text('Coba lagi')),
          ],
        ),
      );
    }
    return RefreshIndicator(
      onRefresh: _muat,
      child: ListView.builder(
        physics: const AlwaysScrollableScrollPhysics(),
        itemCount: _posts.length,
        itemBuilder: (_, i) => ListTile(
          title: Text(_posts[i].judul, maxLines: 1, overflow:
TextOverflow.ellipsis),
          subtitle: Text(_posts[i].isi, maxLines: 2, overflow:
TextOverflow.ellipsis),
          onTap: () => Navigator.push(context,
              MaterialPageRoute(builder: (_) => HalamanDetailPost(post:
```

```
_posts[i]))),
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) => Scaffold(
        appBar: AppBar(title: const Text('Berita')),
        body: _isi(),
      );
}
```

Langkah 3 — Halaman detail. Teruskan objek yang sudah ada, tanpa memanggil API lagi:

```
class HalamanDetailPost extends StatelessWidget {
  final Post post;
  const HalamanDetailPost({super.key, required this.post});

  @override
  Widget build(BuildContext context) => Scaffold(
        appBar: AppBar(title: Text('Post #${post.id}')),
        body: SingleChildScrollView(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(post.judul, style:
Theme.of(context).textTheme.headlineSmall),
              const SizedBox(height: 12),
              Text(post.isi, style: const TextStyle(fontSize: 16, height:
1.5)),
            ],
          ),
        ),
      );
}
```

Langkah 4 — Uji ketahanan. Lakukan berurutan: (a) buka aplikasi dengan internet aktif, (b) tarik daftar ke bawah untuk refresh, (c) aktifkan mode pesawat lalu tutup-buka aplikasi, (d) tekan "Coba lagi" setelah internet dinyalakan. Aplikasi tidak boleh crash di langkah mana pun.

Langkah 5 — Kembangkan fitur orisinal. Contoh fitur pencarian lokal yang bisa ditiru:

```
String _kata = '';
List<Post> get _tersaring => _posts
    .where((p) => p.judul.toLowerCase().contains(_kata.toLowerCase()))
    .toList();

// di AppBar:
bottom: PreferredSize(
  preferredSize: const Size.fromHeight(56),
  child: Padding(
    padding: const EdgeInsets.all(8),
    child: TextField(
      decoration: const InputDecoration(hintText: 'Cari judul...',
prefixIcon: Icon(Icons.search)),
      onChanged: (v) => setState(() => _kata = v),
    ),
  ),
),
```

Ganti `_posts` menjadi `_tersaring` di `ListView.builder`.

Langkah 6 — Pameran singkat. 15 menit terakhir, peserta saling menukar HP dan mencoba aplikasi teman, lalu menjawab checkpoint HOTS di bawah.

Checkpoint HOTS

1. (C4) Bandingkan memanggil API ulang di halaman detail dengan meneruskan objek dari halaman daftar. Kapan masing-masing lebih tepat?
2. (C5) Uji aplikasi teman dengan mematikan internet. Evaluasi cara aplikasinya merespons dan berikan satu rekomendasi perbaikan.
3. (C6) Tambahkan satu fitur orisinal pada aplikasi Anda (pencarian, filter kategori, atau sortir). Jelaskan rancangan dan alasan pemilihannya.

Hari 3: State Management & Final Project Deployment

Hari ketiga merapikan cara aplikasi mengelola data, menuntaskan proyek mandiri, dan menghasilkan file .apk yang siap dibagikan.

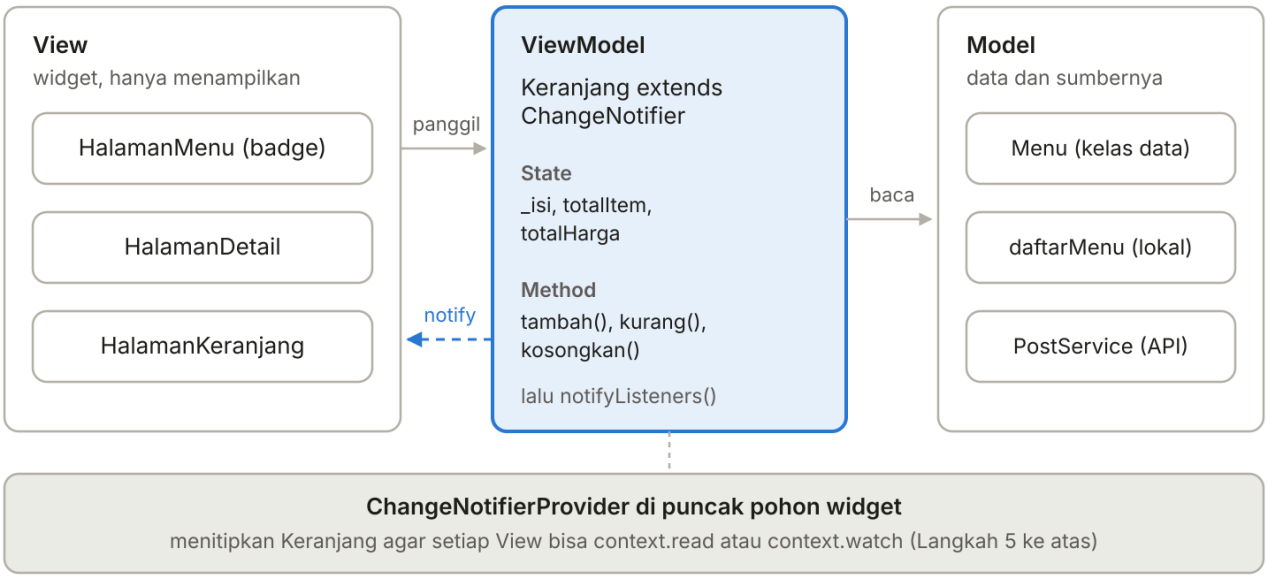
Sesi 1: Pengenalan State Management (Provider / Bloc Dasar)

Tujuan: peserta mampu berbagi state antarhalaman memakai ChangeNotifier dan ListenableBuilder, lalu Provider, dan mengenali kapan Bloc dibutuhkan.

Materi pokok

- Masalah *prop drilling* dan state yang tersebar di banyak `setState`.
- *Ephemeral state* (lokal satu widget) vs *app state* (dipakai banyak halaman).
- Pola MVVM ala Flutter: Model (data dan API), ViewModel (`ChangeNotifier` + `notifyListeners()`), View (widget).
- `ListenableBuilder`: membangun ulang hanya bagian UI yang mendengarkan ViewModel, tanpa paket tambahan.
- Provider sebagai lanjutan: `ChangeNotifierProvider`, `context.watch` vs `context.read`, `Consumer`.
- Gambaran Bloc/Cubit: event, state, dan alur satu arah; perbandingan singkat dengan Provider.

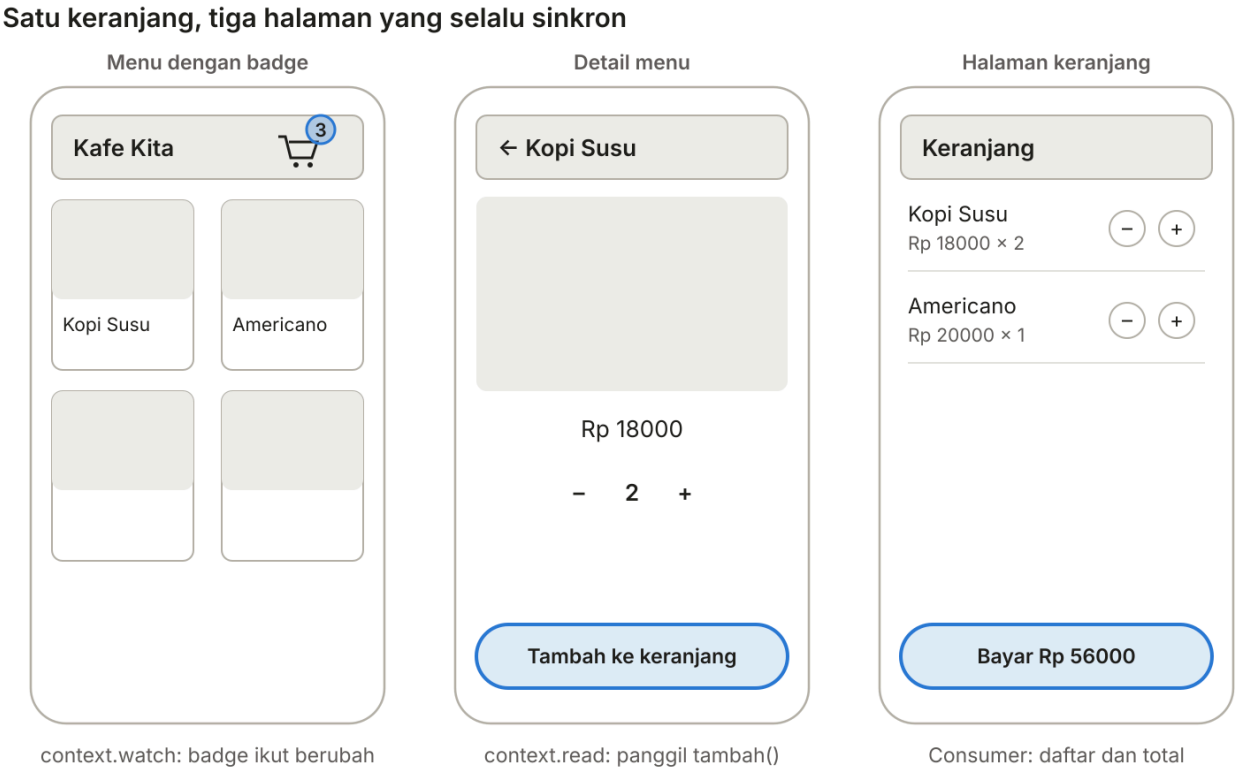
MVVM: View hanya menampilkan, ViewModel memegang state



panggil: View memanggil method ViewModel saat tombol ditekan.
notify: notifyListeners() membuat ListenableBuilder atau context.watch membangun ulang View.

Pola ini sama dengan yang diajarkan Flutter Learning Pathway. Langkah 1–4 tutorial belum memakai pita bawah: Keranjang disimpan di variabel global dan dibaca lewat `ListenableBuilder`; Langkah 5 dan seterusnya menggantinya dengan Provider.

Aktivitas: membuat fitur keranjang belanja pada aplikasi menu kafe; jumlah item tampil di badge AppBar dan halaman keranjang.



Uji keberhasilan: tambah 2 Kopi Susu dan 1 Americano dari layar tengah, lalu pastikan badge menunjukkan 3 dan total di layar kanan Rp 56000.

Tutorial langkah demi langkah

Tutorial ini melanjutkan `kafe_app` dari Hari 2: hasil `Navigator.pop` yang tadinya hanya memunculkan snackbar kini masuk ke keranjang bersama.

Langkah 1 — Rasakan masalahnya dulu. Tanyakan ke peserta: bagaimana halaman Menu, halaman Detail, tab Pesanan, dan badge di AppBar bisa mengetahui isi keranjang yang sama? Tunjukkan bahwa meneruskan data lewat konstruktor ke semua halaman (prop drilling) cepat menjadi rumit.

Langkah 2 — Buat kelas state keranjang (ViewModel).
`lib/viewmodels/keranjang.dart`:

```
import 'package:flutter/foundation.dart';
import '../models/menu.dart';

class Keranjang extends ChangeNotifier {
  final Map<Menu, int> _isi = {};
```

```
  Map<Menu, int> get isi => Map.unmodifiable(_isi);
  int get totalItem => _isi.values.fold(0, (a, b) => a + b);
  int get totalHarga =>
      _isi.entries.fold(0, (a, e) => a + e.key.harga * e.value);

  void tambah(Menu menu, [int jumlah = 1]) {
    _isi.update(menu, (lama) => lama + jumlah, ifAbsent: () => jumlah);
    notifyListeners();
  }

  void kurang(Menu menu) {
    final n = _isi[menu];
    if (n == null) return;
    n > 1 ? _isi[menu] = n - 1 : _isi.remove(menu);
    notifyListeners();
  }

  void kosongkan() {
    _isi.clear();
    notifyListeners();
  }
}
```

Tekankan: data bersifat privat (`_isi`), perubahan hanya lewat method, dan setiap method diakhiri `notifyListeners()`.

Langkah 3 — Pakai tanpa paket dengan ListenableBuilder. Untuk sementara, buat satu instance global di `main.dart`:

```
final keranjang = Keranjang();
```

Di `HalamanDetail`, tombol Pesan langsung memanggil method ViewModel:

```
FilledButton(
  onPressed: () {
    keranjang.tambah(widget.menu, jumlah);
    Navigator.pop(context);
  },
  child: Text('Tambah ke keranjang — Rp ${widget.menu.harga * jumlah}'),
),
```

Di `SliverAppBar` halaman Menu, tampilkan badge yang ikut berubah:

```
actions: [
  ListenableBuilder(
    listenable: keranjang,
    builder: (context, _) => IconButton(
      icon: Badge(
        isLabelVisible: keranjang.totalItem > 0,
        label: Text('${keranjang.totalItem}'),
        child: const Icon(Icons.shopping_cart),
      ),
      onPressed: () {},
    ),
  ),
],
```

Tambahkan menu dari halaman detail: angka badge berubah tanpa `setState`. Inilah pola Flutter Learning Pathway: ViewModel memanggil `notifyListeners()`, dan `ListenableBuilder` membangun ulang hanya bagian yang mendengarkan.

Langkah 4 — Kenali batasnya. Tanyakan ke peserta: bagaimana jika `keranjang` perlu diganti versi palsu saat pengujian, atau ada beberapa ViewModel yang saling bergantung? Variabel global menyulitkan keduanya, sedangkan meneruskan instance lewat konstruktor ke setiap halaman kembali ke masalah prop drilling. Paket `provider` menitipkan ViewModel di pohon widget sehingga halaman mana pun bisa mengambilnya lewat `context`. Kelas `Keranjang` tidak perlu diubah sama sekali.

Langkah 5 — Tambah paket provider.

```
flutter pub add provider
```

Langkah 6 — Pasang provider di puncak aplikasi. Hapus variabel global `keranjang` dari Langkah 3, lalu di `main.dart`:

```
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => Keranjang(),
      child: const AplikasiKafe(),
    ),
  );
}
```

Langkah 7 — Tulis ke keranjang dengan context.read. Di `HalamanDetail`, ubah tombol Pesan:

```
FilledButton(
  onPressed: () {
    context.read<Keranjang>().tambah(widget.menu, jumlah);
    Navigator.pop(context);
  },
  child: Text('Tambah ke keranjang — Rp ${widget.menu.harga * jumlah}'),
),
```

Langkah 8 — Baca dengan context.watch untuk badge. Di `SliverAppBar` halaman Menu, ganti `ListenableBuilder` dari Langkah 3 dengan:

```
actions: [
  Builder(builder: (context) {
    final total = context.watch<Keranjang>().totalItem;
    return IconButton(
      icon: Badge(
        isLabelVisible: total > 0,
        label: Text('$total'),
        child: const Icon(Icons.shopping_cart),
      ),
      onPressed: () {},
    );
  }),
],
```

Tambahkan menu dari halaman detail dan perhatikan angka badge berubah tanpa `setState` apa pun.

Langkah 9 — Halaman keranjang dengan Consumer. Ganti tab "Pesanan" dengan `HalamanKeranjang`:

```
class HalamanKeranjang extends StatelessWidget {
  const HalamanKeranjang({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Keranjang')),
      body: Consumer<Keranjang>(
        builder: (context, k, _) {
```

```
          if (k.isi.isEmpty) return const Center(child: Text('Keranjang
kosong'));
          final item = k.isi.entries.toList();
          return Column(
            children: [
              Expanded(
                child: ListView.builder(
                  itemCount: item.length,
                  itemBuilder: (_, i) {
                    final menu = item[i].key;
                    return ListTile(
                      title: Text(menu.nama),
                      subtitle: Text('Rp ${menu.harga} × ${item[i].value}'),
                      trailing: Row(mainAxisSize: MainAxisSize.min, children:
[
                        IconButton(icon: const Icon(Icons.remove), onPressed:
() => k.kurang(menu)),
                        IconButton(icon: const Icon(Icons.add), onPressed: ()
=> k.tambah(menu)),
                      ]),
                    );
                  },
                ),
              ),
              Padding(
                padding: const EdgeInsets.all(16),
                child: FilledButton(
                  onPressed: k.kosongkan,
                  child: Text('Bayar Rp ${k.totalHarga}'),
                ),
              ),
            ],
          );
        },
      ),
    );
  }
}
```

Langkah 10 — Demonstrasi kesalahan watch vs read. Ganti `context.read` di tombol detail menjadi `context.watch` dan tunjukkan pesan error/peringatan yang muncul. Jelaskan aturan praktis: `watch` di dalam `build`, `read` di dalam fungsi event seperti

`onPressed`.

Langkah 11 — Sekilas Bloc/Cubit (demo fasilitator, 10 menit). Tunjukkan padanan sederhana dengan Cubit agar peserta melihat polanya:

```
// flutter pub add flutter_bloc
class JumlahCubit extends Cubit<int> {
  JumlahCubit() : super(0);
  void tambah() => emit(state + 1);
}
// di UI: BlocBuilder<JumlahCubit, int>(builder: (_, n) => Text('$n'))
```

Bandingkan: Provider mengubah data lalu memberi tahu; Bloc/Cubit memancarkan state baru yang utuh setiap kali berubah.

Langkah 12 — Tantangan mandiri. Tambahkan ViewModel kedua, `Favorit`, dan pasang keduanya memakai `MultiProvider`. Tombol hati di `KartuMenu` menandai menu favorit.

Checkpoint HOTS

1. (C4) Klasifikasikan state berikut sebagai ephemeral atau app state: status login, tab yang aktif, isi keranjang, teks di kolom pencarian, tema gelap/terang. Jelaskan alasan tiap klasifikasi.
2. (C4) Bandingkan `context.watch` dan `context.read`. Apa yang terjadi bila `watch` dipakai di dalam fungsi `onPressed`?
3. (C5) Tim Anda terdiri dari 2 orang dan aplikasinya kecil (5 halaman). Evaluasi apakah lebih tepat memakai Provider atau Bloc. Bagaimana jawabannya berubah untuk tim 10 orang dengan 50 halaman?
4. (C5) Seorang peserta menaruh seluruh data aplikasi dalam satu `ChangeNotifier` raksasa. Nilailah dampaknya terhadap performa dan keterawatan kode.
5. (C6) Rancang struktur state (kelas, data, dan method) untuk aplikasi to-do list dengan fitur tambah, tandai selesai, hapus, dan filter.

Sesi 2: Finishing Proyek Aplikasi Mandiri

Tujuan: peserta menyelesaikan aplikasi mandiri yang menggabungkan materi hari 1–3.

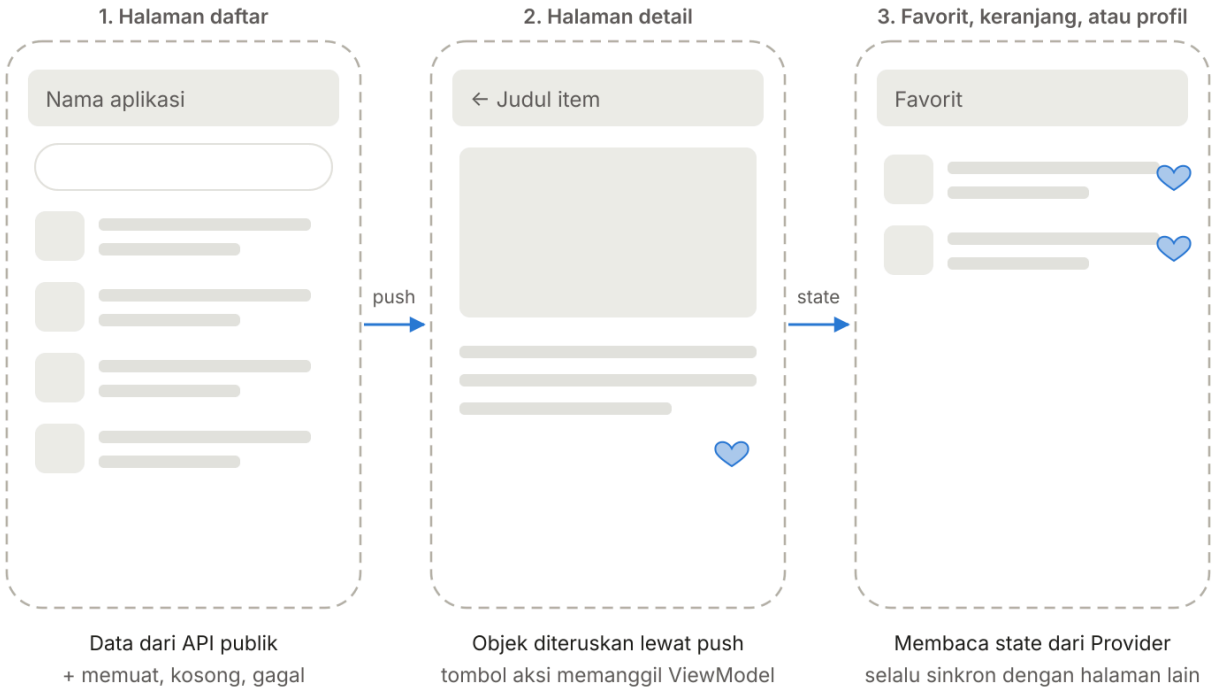
Syarat minimal proyek

- Minimal 3 halaman dengan navigasi.
- Mengambil data dari API publik (atau data lokal bila tidak memungkinkan).
- Memakai Provider untuk minimal satu fitur (favorit, keranjang, atau filter).

- Menangani keadaan memuat, kosong, dan gagal.

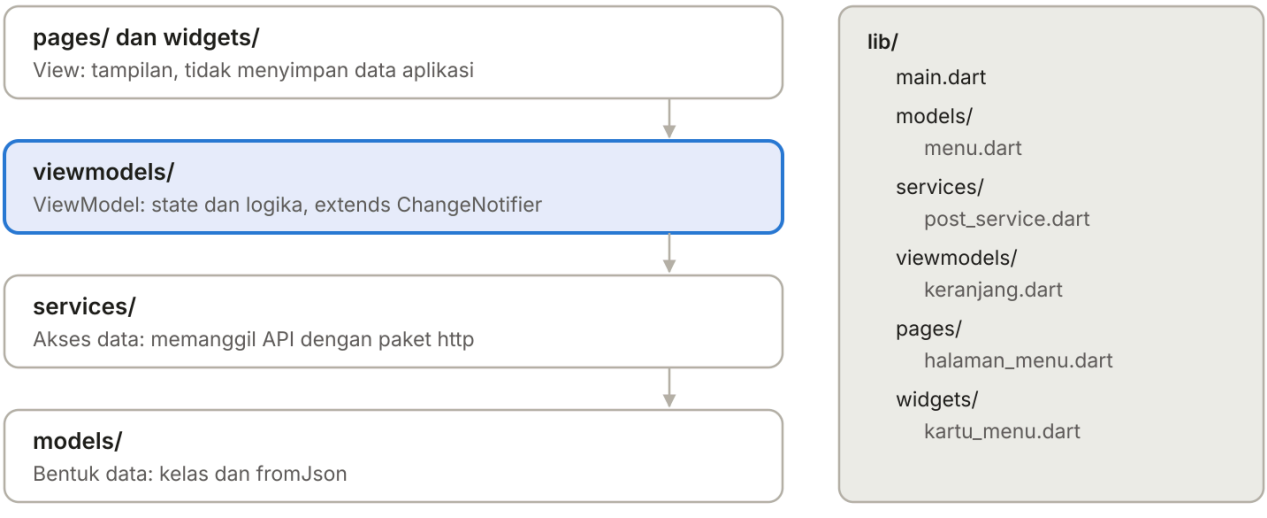
Alur kerja: 15 menit menulis rencana (fitur inti dan sketsa halaman), 75 menit pengerjaan dengan pendampingan fasilitator, 15 menit *code review* berpasangan.

Kerangka minimal proyek mandiri: tiga halaman



Peserta menyalin kerangka ini ke kertas pada Langkah 2, lalu mengganti isi setiap kotak dengan tema aplikasinya sendiri.

Folder proyek mengikuti lapisan MVVM



Panah = memanggil. Lapis atas boleh memakai lapis bawah, tidak sebaliknya.

Saat code review di Langkah 7, cek arah panah: halaman boleh memanggil ViewModel, tetapi ViewModel tidak boleh mengimpor widget.

Panduan langkah demi langkah

Sesi ini berupa kerja mandiri. Langkah di bawah dibacakan atau ditayangkan di awal, lalu fasilitator berkeliling.

Langkah 1 — Pilih tema (5 menit). Peserta boleh melanjutkan `kafe_app` atau `berita_app`, atau memilih tema baru dengan API gratis, misalnya:

- Katalog produk: `https://fakestoreapi.com/products`
- Ensiklopedia Pokémon: `https://pokeapi.co/api/v2/pokemon`
- Daftar resep: `https://www.themealdb.com/api/json/v1/1/search.php?s=chicken`

Langkah 2 — Tulis rencana satu halaman (10 menit). Di kertas atau catatan, isi empat hal: (1) nama aplikasi dan penggunanya, (2) tiga halaman dan sketsa kasarnya, (3) data apa yang diambil dari API, (4) fitur apa yang memakai Provider. Fasilitator memeriksa rencana sebelum peserta mulai coding.

Langkah 3 — Siapkan struktur folder (5 menit).

```
lib/
  main.dart
  models/       // kelas data + fromJson
  services/     // pemanggilan API
  viewmodels/   // ChangeNotifier (ViewModel)
  pages/        // satu file per halaman
  widgets/      // komponen yang dipakai ulang
```

Langkah 4 — Kerjakan dari data ke tampilan (60 menit). Urutan yang disarankan, tiap langkah dijalankan dan dicek sebelum lanjut:

1. Model + service, uji dengan `print` hasil API.
2. Halaman daftar dengan keadaan memuat/gagal (pola Praktik Hari 2).
3. Halaman detail + navigasi.
4. Provider untuk fitur favorit/keranjang/filter.
5. Halaman ketiga (misalnya favorit, keranjang, atau profil).
6. Poles tampilan: tema warna, ikon, nama aplikasi.

Langkah 5 — Simpan progres berkala. Bila peserta memakai Git:

```
git init
git add .
git commit -m "halaman daftar selesai"
```

Commit setiap kali satu langkah di atas selesai, agar mudah kembali bila ada yang rusak.

Langkah 6 — Aturan saat macet. Bila terjebak lebih dari 10 menit: (1) baca pesan error dari baris paling atas, (2) tanya teman sebelah, (3) baru angkat tangan memanggil fasilitator. Fitur yang tidak sempat selesai dicatat sebagai rencana versi 2.0, bukan dipaksakan.

Langkah 7 — Code review berpasangan (15 menit). Pasangan saling membuka kode dan memeriksa empat hal: (1) apakah ada widget yang terlalu panjang (> 80 baris) dan bisa dipecah, (2) apakah ada `print` atau kode coba-coba yang tertinggal, (3) apakah aplikasi tetap aman saat internet mati, (4) apakah penamaan variabel jelas. Catat dua masukan untuk pasangan.

Checkpoint HOTS

1. (C4) Waktu tinggal 30 menit dan masih ada 4 fitur belum selesai. Analisis fitur mana yang Anda prioritaskan dan mana yang ditunda, beserta kriterianya.
2. (C5) Lakukan code review pada proyek teman: temukan satu bagian kode yang bisa dipecah menjadi widget terpisah dan jelaskan manfaatnya.
3. (C5) Evaluasi proyek Anda sendiri terhadap syarat minimal di atas. Bagian mana yang paling rapuh bila aplikasi dipakai 1.000 orang?
4. (C6) Rumuskan *roadmap* versi 2.0 aplikasi Anda: tiga fitur berikutnya dan alasan bisnis atau pengguna di balik tiap fitur.

Sesi 3: Build Aplikasi Menjadi .apk dan Tips Publikasi

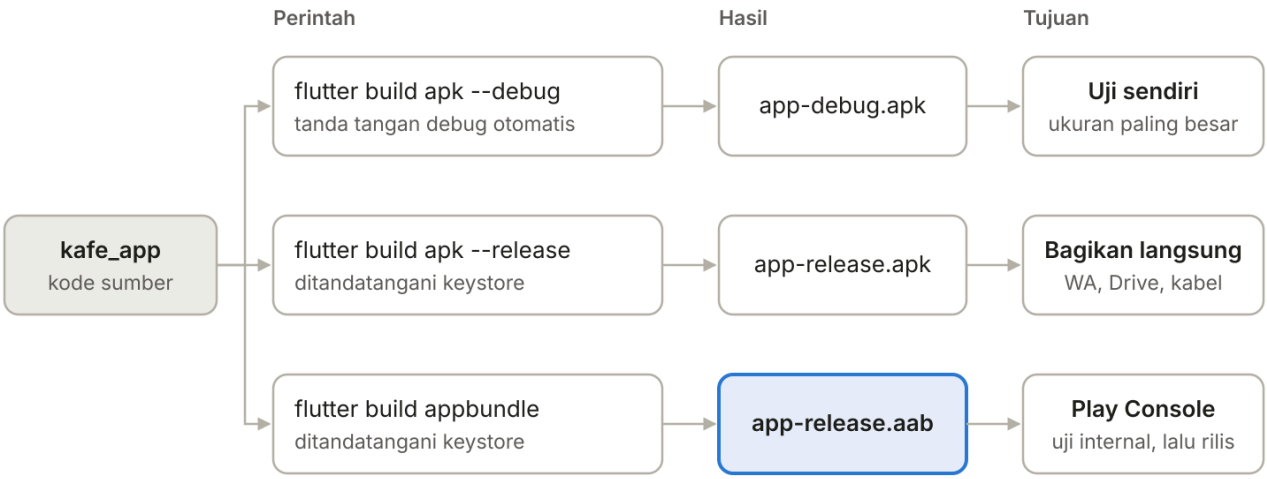
Tujuan: peserta mampu menghasilkan file .apk rilis dan memahami langkah publikasi ke Google Play.

Materi pokok

- Mode *debug*, *profile*, dan *release*; perbedaan ukuran dan performanya.
- Mengganti nama aplikasi, ikon (paket `flutter_launcher_icons`), dan `applicationId`.
- Membuat *keystore* dan konfigurasi *signing*; pentingnya menyimpan keystore dengan aman.
- Perintah `flutter build apk --release`, `--split-per-abi`, dan `flutter build appbundle` (.aab).

- Alur Google Play Console: akun developer, listing toko, kebijakan privasi, pengujian internal/tertutup, dan review.

Satu kode, tiga hasil build untuk tujuan berbeda

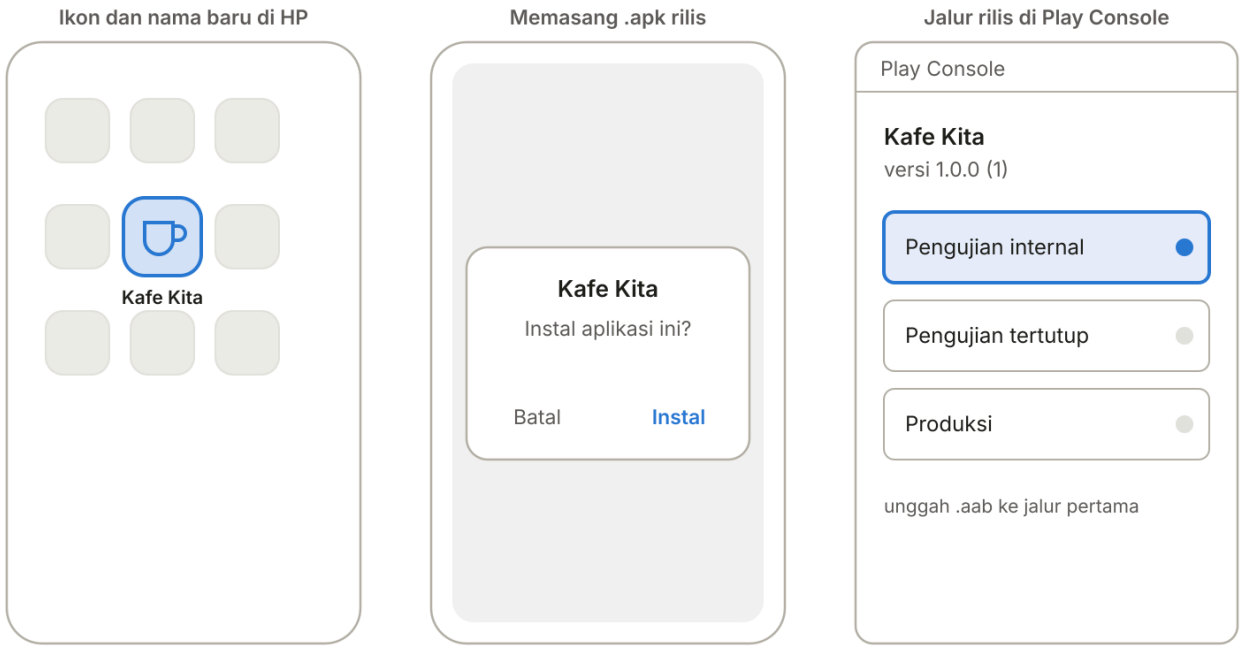


> Keystore yang sama wajib dipakai untuk setiap pembaruan aplikasi.
> Simpan file .jks dan password-nya di luar folder proyek, misalnya di pengelola password.

Peserta memilih jalur sesuai tujuan: .apk untuk dibagikan langsung saat pelatihan, .aab bila nanti mengunggah ke Google Play.

Aktivitas: setiap peserta mem-build .apk rilis dan memasangnya di HP sendiri atau HP teman.

Hasil akhir: ikon di HP dan rilis di jalur pengujian



Dua layar kiri wajib dicapai setiap peserta; layar kanan cukup didemonstrasikan fasilitator pada Langkah 9.

Tutorial langkah demi langkah

Catatan fasilitator: proyek Flutter baru memakai file Gradle berformat Kotlin (`build.gradle.kts`). Bila proyek peserta masih `build.gradle` (Groovy), sintaksnya sedikit berbeda; rujuk docs.flutter.dev/deployment/android.

Langkah 1 — Ganti nama aplikasi. Buka `android/app/src/main/AndroidManifest.xml`, ubah `android:label="kafe_app"` menjadi `android:label="Kafe Kita"`. Pastikan juga izin `INTERNET` dari Hari 2 sudah ada.

Langkah 2 — Ganti ikon aplikasi. Siapkan gambar persegi 1024×1024 px di `assets/icon.png`, lalu:

```
flutter pub add dev:flutter_launcher_icons
```

Tambahkan di `pubspec.yaml` (sejajar dengan `dependencies`):

```
flutter_launcher_icons:
  android: true
  ios: false
  image_path: "assets/icon.png"
```

Jalankan `dart run flutter_launcher_icons`.

Langkah 3 — Atur applicationId dan versi. Di `android/app/build.gradle.kts`, ganti `applicationId = "com.example.kafe_app"` menjadi ID unik, misalnya `"id.ac.amikom.kafekita"`. ID ini tidak bisa diubah setelah terbit di Play Store. Versi diatur di `pubspec.yaml` lewat baris `version: 1.0.0+1` (nama versi + nomor build).

Langkah 4 — Build .apk pertama dan bandingkan ukuran.

```
flutter build apk --debug
flutter build apk --release
flutter build apk --release --split-per-abi
```

Hasilnya ada di `build/app/outputs/flutter-apk/`. Catat ukuran ketiganya di papan tulis untuk bahan checkpoint HOTS.

Langkah 5 — Buat keystore. Keystore adalah "tanda tangan" pengembang. Jalankan (keytool ikut terpasang bersama Android Studio):

```
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA \
  -keysize 2048 -validity 10000 -alias upload
```

Isi password dan identitas. Simpan file `.jks` dan password-nya di tempat aman (misalnya pengelola password), jangan di dalam folder proyek.

Langkah 6 — Hubungkan keystore ke proyek. Buat `android/key.properties`:

```
storePassword=passwordAnda
keyPassword=passwordAnda
keyAlias=upload
storeFile=/home/nama/upload-keystore.jks
```

Tambahkan `key.properties` dan `*.jks` ke `.gitignore`. Lalu di `android/app/build.gradle.kts`:

```
import java.util.Properties
import java.io.FileInputStream

val keystoreProperties = Properties()
val keystorePropertiesFile = rootProject.file("key.properties")
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(FileInputStream(keystorePropertiesFile))
}

android {
    // ...
    signingConfigs {
        create("release") {
            keyAlias = keystoreProperties["keyAlias"] as String
            keyPassword = keystoreProperties["keyPassword"] as String
            storeFile = keystoreProperties["storeFile"]?.let { file(it) }
            storePassword = keystoreProperties["storePassword"] as String
        }
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
        }
    }
}
```

Langkah 7 — Build rilis bertanda tangan.

```
flutter clean
flutter build apk --release        # untuk dibagikan langsung
flutter build appbundle --release  # .aab untuk Google Play
```

File .aab ada di `build/app/outputs/bundle/release/`.

Langkah 8 — Pasang dan uji di HP. Dengan HP tersambung, jalankan `flutter install`, atau kirim file .apk ke HP (via kabel, Google Drive, atau WhatsApp) lalu buka dan izinkan instalasi dari sumber tak dikenal. Uji ulang semua fitur, terutama pengambilan data dari internet.

Langkah 9 — Tur Google Play Console (demo fasilitator, 15 menit). Tunjukkan alur: daftar akun developer (biaya sekali bayar), *Create app*, isi *store listing* (judul, deskripsi, ikon 512 px, minimal 2 screenshot, grafik fitur 1024×500), tautan kebijakan privasi, kuesioner rating konten, unggah .aab ke jalur *Internal testing*, lalu kirim untuk review. Sebutkan bahwa akun personal baru biasanya diwajibkan menjalani pengujian tertutup bersama sejumlah penguji selama beberapa waktu sebelum bisa rilis produksi; cek persyaratan terbaru di Play Console karena kebijakannya berubah dari waktu ke waktu.

Checkpoint HOTS

1. (C4) Bandingkan ukuran .apk debug dan release dari proyek Anda. Analisis faktor yang membuat keduanya berbeda.
2. (C4) Mengapa Google Play mensyaratkan format .aab, sementara .apk masih dipakai untuk distribusi langsung? Uraikan perbedaan cara keduanya sampai ke perangkat pengguna.
3. (C5) Seorang developer kehilangan file keystore setelah aplikasinya rilis. Evaluasi dampaknya dan kebiasaan apa yang seharusnya diterapkan sejak awal.
4. (C6) Susun daftar periksa (checklist) pra-rilis untuk aplikasi Anda: minimal 6 butir mencakup teknis, tampilan, dan legal.

Penutupan: Review Proyek Terbaik dan Pembagian Sertifikat

Alur: setiap peserta (atau 5–6 peserta terpilih bila waktu terbatas) mendemokan aplikasinya 3 menit, dinilai dengan rubrik di Lampiran, lalu fasilitator mengumumkan proyek terbaik dan membagikan sertifikat.

Panduan langkah demi langkah

Langkah 1 — Persiapan (fasilitator, sebelum 14.30). Siapkan proyektor atau aplikasi *screen mirroring* (misalnya scrcpy) agar layar HP tampil di depan, lembar rubrik dari Lampiran untuk setiap juri, dan sertifikat yang sudah dicetak atau versi digitalnya.

Langkah 2 — Susun urutan demo. Undi urutan presentasi. Bila peserta lebih dari 15 orang, pilih 5–6 proyek berdasarkan hasil code review Sesi 2, dan sisanya mengunggah video demo 1 menit ke folder bersama.

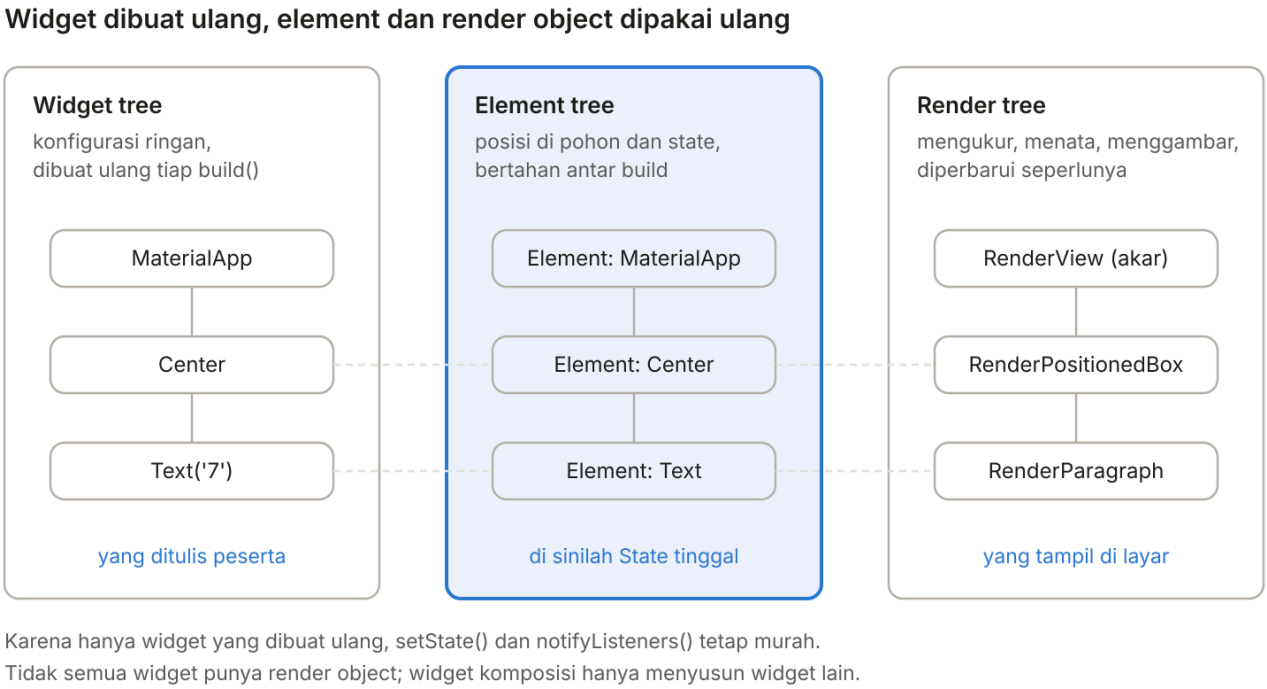
Langkah 3 — Format demo 3 menit per peserta.

1. Menit 1: masalah yang diselesaikan dan siapa penggunanya.
2. Menit 2: demo langsung alur utama dari .apk rilis (bukan mode debug).
3. Menit 3: satu keputusan teknis yang paling sulit dan alasannya.

Sediakan 1 menit tanya jawab; juri mengisi rubrik selama demo berlangsung.

Langkah 4 — Rekap nilai. Jumlahkan skor rubrik (nilai = jumlah skor × bobot ÷ 4 × 100). Bila ada nilai sama, juri memilih berdasarkan kriteria Fungsionalitas.

Sisipan (15 menit) — Bagaimana Flutter bekerja. Sambil juri merekap nilai, tutup materi dengan gambaran besar dari langkah terakhir Pathway, How Flutter works. Gambarkan di papan tiga lapis pohon: *widget tree* (konfigurasi ringan yang ditulis peserta), *element tree* (yang menyimpan posisi dan state), dan *render tree* (yang mengukur, menata, dan menggambar). Kaitkan dengan pengalaman tiga hari: mengapa `setState` dan `notifyListeners()` murah, mengapa `const` membantu performa, dan mengapa error "unbounded height" muncul saat pengukuran ukuran.



Garis putus-putus menunjukkan padanan antarpohon; tekankan bahwa peserta selama tiga hari hanya menulis kolom kiri, dan Flutter mengurus dua kolom lainnya.

Langkah 5 — Refleksi bersama (15 menit). Peserta menjawab checkpoint HOTS refleksi di bawah secara tertulis, lalu 3 peserta membagikan jawabannya.

Langkah 6 — Pengumuman dan sertifikat. Umumkan proyek terbaik beserta alasan singkat dari juri, bagikan sertifikat, dan bagikan tautan sumber belajar lanjutan: docs.flutter.dev, pub.dev, dan kanal YouTube resmi Flutter.

Langkah 7 — Evaluasi pelatihan. Bagikan formulir umpan balik singkat (kepuasan materi, kecepatan penyampaian, saran) sebelum peserta pulang.

Checkpoint HOTS (refleksi)

1. (C4) Dari semua materi tiga hari ini, konsep mana yang paling mengubah cara Anda berpikir tentang membangun aplikasi? Jelaskan sebelum dan sesudahnya.
2. (C5) Bandingkan proyek terbaik dengan proyek Anda sendiri. Apa satu keputusan desain atau teknis mereka yang akan Anda tiru, dan mengapa?
3. (C6) Rancang rencana belajar 30 hari ke depan untuk melanjutkan pelatihan ini, lengkap dengan target aplikasi yang ingin Anda rilis.

Lampiran: Rubrik Penilaian Proyek Akhir

Proyek dinilai pada skala 1–4 per kriteria; nilai akhir = jumlah (skor × bobot) ÷ 4 × 100.

Kriteria	Bobot	4 (Sangat baik)	3 (Baik)	2 (Cukup)	1 (Kurang)
Fungsionalitas	30%	Semua syarat minimal berjalan tanpa error	Syarat terpenuhi, ada bug kecil	Sebagian syarat belum berjalan	Aplikasi sering error atau tidak berjalan
Integrasi API & state	25%	Data API, model, dan Provider dipakai dengan tepat	API dan Provider dipakai, struktur kurang rapi	Hanya salah satu yang diterapkan	Belum ada integrasi API maupun state
UI/UX	20%	Rapi, konsisten, keadaan memuat/kosong/gagal ditangani	Rapi, sebagian keadaan belum ditangani	Ada overflow atau tampilan tidak konsisten	Sulit dipakai
Kualitas kode	15%	Widget terpecah rapi, penamaan jelas	Cukup rapi, ada bagian yang menumpuk	Banyak kode berulang	Sulit dibaca
Presentasi	10%	Demo lancar, mampu menjelaskan keputusan teknis	Demo lancar, penjelasan terbatas	Demo tersendat	Tidak dapat mendemokan